Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

You are given a graph, and your task is to find the minimum vertex cover for the given graph. A vertex cover is a subset of vertices in the graph such that every edge in the graph is incident to at least one vertex in the cover.

Input Format

The first line contains an integer 'vertices' representing the number of vertices in the graph.

The second line contains an integer 'edges' representing the number of edges in the graph.

The following 'edges' lines each contain two integers 'u' and 'v' representing an edge between vertices 'u' and 'v'.

Output Format

The output prints the minimum vertex cover for the given graph, where each vertex in the cover is separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7
6
0 1
0 2
1 3
3 4
4 5
5 6

Output: 0 1 3 4 5 6

Answer

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int vertices, edges;

    // Read number of vertices and edges
    if (scanf("%d", &vertices) != 1) return 0;
    if (scanf("%d", &edges) != 1) return 0;

    int edge_u[501], edge_v[501];
    bool visited[101] = {false};

    // Store edges
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &edge_u[i], &edge_v[i]);
    }

    // Vertex Cover logic
    // We traverse edges and if an edge is not covered, we add its vertices
```

```
    for (int i = 0; i < edges; i++) {
        int u = edge_u[i];
        int v = edge_v[i];

        // If neither vertex u nor vertex v is in the vertex cover
        if (!visited[u] && !visited[v]) {
            visited[u] = true;
            visited[v] = true;
        }
    }

    // Print the vertices in the cover
    bool first = true;
    for (int i = 0; i < vertices; i++) {
        if (visited[i]) {
            if (!first) printf(" ");
            printf("%d", i);
            first = false;
        }
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

A financial analyst is studying the daily profit and loss values of a company over a period of time. Each day's profit or loss is recorded as an integer in an array, where positive values indicate profit and negative values indicate loss.

The analyst wants to identify a continuous period of days during which the company achieved the maximum total profit. This period must consist of consecutive days only, and at least one day must be selected.

Given an array of integers representing daily profit and loss values, determine the maximum possible sum of any contiguous subarray.

To solve this efficiently, you must use a linear-time algorithm.

Input Format

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing daily profit or loss values.

Output Format

Print a single integer representing the maximum sum of a contiguous subarray.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9
-2 1 -3 4 -1 2 1 -5 4
Output: 6

Answer

```
#include <stdio.h>

int main() {
    int n;
    // Read the number of days
    if (scanf("%d", &n) != 1) return 0;

    long long current_max = 0;
    long long global_max = -2147483648; // Minimum possible value for a 32-bit
int

    for (int i = 0; i < n; i++) {
        int daily_value;
        scanf("%d", &daily_value);

        if (i == 0) {
            // Initialize with the first element
            current_max = daily_value;
```

```
            global_max = daily_value;
        } else {
            // Decide: extend the existing subarray or start a new one from today?
            // current_max = max(daily_value, current_max + daily_value)
            if (daily_value > (current_max + daily_value)) {
                current_max = daily_value;
            } else {
                current_max = current_max + daily_value;
            }

            // Update global_max if the current path is the best seen so far
            if (current_max > global_max) {
                global_max = current_max;
            }
        }
    }

    // Print the result without trailing spaces or new lines
    printf("%lld", global_max);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Given an array of integers, determine whether there exists a majority element in the array.

A majority element is defined as an element that appears more than n/2 times in the array.

Your task is to implement a function majorityElement() that takes the input array and returns the majority element if it exists. If no such element exists, return -1.

Formula

An element is considered a majority element if:

frequency(element) > n/2

where n is the total number of elements in the array.

Input Format

The first line of input contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers a1 a2 a3 ... an representing the array elements.

Output Format

Print the majority element if it exists. Otherwise, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9
3 3 4 2 4 4 2 4 4

Output: 4

Answer

```
#include <stdio.h>

int main() {
    int n;
    // Read the size of the array
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Phase 1: Find a candidate using Boyer-Moore Voting Algorithm
```

```
    int candidate = -1, votes = 0;
    for (int i = 0; i < n; i++) {
        if (votes == 0) {
            candidate = arr[i];
            votes = 1;
        } else if (arr[i] == candidate) {
            votes++;
        } else {
            votes--;
        }
    }

    // Phase 2: Verify if the candidate is actually the majority element
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == candidate) {
            count++;
        }
    }

    // Output the result based on the n/2 condition
    if (count > n / 2) {
        printf("%d", candidate);
    } else {
        printf("-1");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Aarav is analyzing temperature fluctuations recorded over several days. The temperature changes are stored in an array of integers. He wants to identify a contiguous period of days such that the product of the temperature changes during that period is maximized.

Given an array of integers, determine the maximum product that can be

obtained from any contiguous subarray containing at least one element.

Your task is to compute this value efficiently.

Formula

To correctly handle both positive and negative values, track both the maximum and minimum product ending at each index.

If the current element is negative, swap the previous maximum and minimum values before updating.

At each index i:

currentMax = max(arr[i], arr[i] × currentMax)

currentMin = min(arr[i], arr[i] × currentMin)

The final answer is the maximum value obtained among all currentMax values.

Input Format

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Output Format

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 3 -2 4 -1 5
Output: 240

Answer

```
#include <stdio.h>

// Helper function to find maximum of two long long integers
long long max(long long a, long long b) {
    return (a > b) ? a : b;
}

// Helper function to find minimum of two long long integers
long long min(long long a, long long b) {
    return (a < b) ? a : b;
}

int main() {
    int n;
    // Read the number of days
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Initialize trackers with the first element
    long long currentMax = arr[0];
    long long currentMin = arr[0];
    long long globalMax = arr[0];

    for (int i = 1; i < n; i++) {
        // If the current element is negative, currentMax and currentMin swap roles
        if (arr[i] < 0) {
            long long temp = currentMax;
            currentMax = currentMin;
            currentMin = temp;
        }

        // Standard update based on the formula provided
        currentMax = max((long long)arr[i], currentMax * arr[i]);
        currentMin = min((long long)arr[i], currentMin * arr[i]);

        // Update global max if currentMax is higher
        if (currentMax > globalMax) {
```

```
            globalMax = currentMax;
        }
    }

    // Print only the final maximum product
    printf("%lld", globalMax);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Sidhant is a logistics manager responsible for planning delivery routes for a fleet of trucks. He has a list of cities, and the distances between every pair of cities are given in the form of a matrix.

Each truck must start from a central warehouse (city 0), visit every other city exactly once, and return back to the warehouse. The goal is to minimize the total travel distance.

Sidhant needs to determine the minimum distance required to complete the route and also print the optimal path taken.

Input Format

The first line contains an integer n, representing the number of cities.

The next n lines contain n space-separated integers, representing the distance matrix.

Output Format

The first line of output prints the minimum travel distance.

In the next line, print the optimal path starting and ending at city 0.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: 80
0 1 3 2 0

Answer

```
#include <stdio.h>
#include <string.h>

#define INF 1e9

int n;
int dist[15][15];
int dp[1 << 15][15];
int parent[1 << 15][15];

// TSP function using DP + Bitmasking
int tsp(int mask, int pos) {
    if (mask == (1 << n) - 1) {
        return dist[pos][0];
    }
    if (dp[mask][pos] != -1) {
        return dp[mask][pos];
    }

    int ans = INF;
    for (int city = 0; city < n; city++) {
        if ((mask & (1 << city)) == 0) {
            int newAns = dist[pos][city] + tsp(mask | (1 << city), city);
            if (newAns < ans) {
                ans = newAns;
                parent[mask][pos] = city;
            }
        }
    }
    return dp[mask][pos] = ans;
```

```
}

// Function to print the path
void printPath() {
    int mask = 1;
    int pos = 0;
    printf("0");
    while (mask != (1 << n) - 1) {
        pos = parent[mask][pos];
        mask |= (1 << pos);
        printf(" %d", pos);
    }
    printf(" 0");
}

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }

    // Initialize DP table
    memset(dp, -1, sizeof(dp));
    memset(parent, -1, sizeof(parent));

    // Find min distance starting from city 0
    int minDistance = tsp(1, 0);

    // Output formatted results
    printf("%d\n", minDistance);
    printPath();

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 49

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items such that their total weight is exactly equal to a given target weight. Each item has a known weight, and every item can be selected at most once.

Given a list of item weights and a target weight k, determine whether there exists a subset of items whose total weight is exactly k.

If such a subset exists, print any one valid subset.

If no such subset exists, print -1.

The solution must be implemented using Dynamic Programming.

Formula

Let

dp[i][j] = true

if a subset of the first i items can form a sum j.

Transition:

If arr[i-1] > j

dp[i][j] = dp[i-1][j]

Else

dp[i][j] = dp[i-1][j] OR dp[i-1][j - arr[i-1]]

Input Format

The first line contains two integers n and k, where n is the number of items and k is the target weight.

The second line contains n space-separated integers representing item weights.

Output Format

If a valid subset exists, print the elements of one such subset in a single line separated by spaces.

If no subset exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 9
3 34 4 12 5
Output: 5 4

Answer

```
#include <stdio.h>
#include <stdbool.h>

void findSubset() {
    int n, k;
    if (scanf("%d %d", &n, &k) != 2) return;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // dp[i][j] will be true if a sum j is possible with first i elements
    bool dp[n + 1][k + 1];

    // Initialization
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= k; j++) {
            if (j == 0) dp[i][j] = true;
            else dp[i][j] = false;
        }
    }

    // Filling the DP table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            if (arr[i - 1] > j) {
                dp[i][j] = dp[i - 1][j];
            } else {
                dp[i][j] = dp[i - 1][j] || dp[i - 1][j - arr[i - 1]];
            }
        }
    }

    // If target sum is not possible
    if (!dp[n][k]) {
        printf("-1");
```

```
        return;
    }

    // Backtracking to find the items
    int curr_sum = k;
    bool first = true;
    for (int i = n; i > 0 && curr_sum > 0; i--) {
        // If the sum was possible without the current item, skip it
        // Otherwise, it was included
        if (dp[i][k] && !dp[i - 1][curr_sum]) {
            if (!first) printf(" ");
            printf("%d", arr[i - 1]);
            curr_sum -= arr[i - 1];
            first = false;
        } else if (curr_sum >= arr[i-1] && dp[i-1][curr_sum - arr[i-1]]) {
            // This item was part of the subset
            if (!first) printf(" ");
            printf("%d", arr[i - 1]);
            curr_sum -= arr[i - 1];
            first = false;
        }
    }
}

int main() {
    findSubset();
    return 0;
}
```

Status : Partially correct **Marks : 9/10**

2. Problem Statement

Ramya is working as a logistics coordinator for a delivery company. Every day, she receives a list of delivery locations arranged along a straight highway. Each location has a certain delivery priority value (which may be positive or negative depending on profit or loss).

Ramya wants to choose a continuous sequence of delivery locations such that the total delivery priority is maximized, ensuring maximum profit for

that day.

Since the company handles a large number of deliveries, Ramya needs an efficient solution that runs in polynomial time.

Input Format

The first line contains an integer n, representing the number of delivery locations.

The second line contains n space-separated integers, representing the priority values.

Output Format

The output prints a single integer representing the maximum total priority of any contiguous sequence of locations.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8
-2 3 5 -1 4 -5 2 6
Output: 14

Answer

```
#include <stdio.h>

int main() {
    int n;
    // Read the number of delivery locations
    if (scanf("%d", &n) != 1) return 0;

    long long current_max = 0;
    long long global_max = -2147483648; // Minimum possible 32-bit integer

    for (int i = 0; i < n; i++) {
        int priority;
        scanf("%d", &priority);

        // Update the current maximum subarray sum ending at this index
```

```
        if (i == 0) {
            current_max = priority;
            global_max = priority;
        } else {
            // Either start a new subarray at this element or extend the previous one
            current_max = (priority > (current_max + priority)) ? priority : (current_max
+ priority);

            // Update the global maximum if the current one is higher
            if (current_max > global_max) {
                global_max = current_max;
            }
        }
    }

    // Print the final result
    printf("%lld", global_max);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Ragav is managing a chain of warehouses located along a long highway. Each warehouse is positioned at a specific coordinate on the road.

To improve efficiency, Ragav wants to choose a central hub location such that the total travel distance from all warehouses to this hub is minimized.

He can place the hub at any one of the existing warehouse locations.

Since the number of warehouses can be very large, Ragav needs an efficient algorithm that runs in polynomial time to compute the optimal location.

Input Format

The first line contains an integer n, representing the number of warehouses.

The second line contains n space-separated integers, representing the positions of warehouses on the highway.

Output Format

The output prints a single integer representing the minimum total distance from all warehouses to the chosen hub.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
1 2 3 6 10
Output: 13

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort
int compare(const void *a, const void *b) {
    long long x = *(long long *)a;
    long long y = *(long long *)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    long long *positions = (long long *)malloc(n * sizeof(long long));
    for (int i = 0; i < n; i++) {
        scanf("%lld", &positions[i]);
    }
    // 1. Sort positions to find the median
    qsort(positions, n, sizeof(long long), compare);
```

```
    // 2. Identify the median
    // For both even and odd n, the element at index n/2 works as an optimal hub
    long long hub = positions[n / 2];

    // 3. Calculate total distance to the hub
    long long total_distance = 0;
    for (int i = 0; i < n; i++) {
        long long diff = positions[i] - hub;
        if (diff < 0) diff = -diff; // Absolute value
        total_distance += diff;
    }

    // 4. Output the result
    printf("%lld", total_distance);

    free(positions);
    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Peter is a project manager in a software company. He has a list of employees, and each employee has a certain skill contribution value (which can be positive or negative depending on their impact on the project).

Peter wants to form a team such that the total skill contribution is exactly equal to a given target value. Each employee can be selected at most once, and the team must contain at least one employee.

Since the number of possible teams grows rapidly with the number of employees, Peter needs to determine whether such a team exists.

Input Format

The first line contains an integer n, representing the number of employees.

The second line contains n space-separated integers, representing the skill contribution of each employee.

The third line contains an integer target, representing the required total skill.

Output Format

If a valid team exists, the output prints: "Team Found: x1 x2 x3 ..."

If no such team exists, the output prints: "No valid team found"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
3 7 -2 5 1
6
Output: Team Found: 3 -2 5

Answer

```
#include <stdio.h>
#include <stdbool.h>

int n, target;
int skills[20];
int team[20];
bool found = false;

// Backtracking function to find a subset that sums to target
void findTeam(int index, int currentSum, int teamSize) {
    if (found) return;

    // Check if the current sum matches target (team must have at least one
member)
    if (teamSize > 0 && currentSum == target) {
        found = true;
        printf("Team Found: ");
        for (int i = 0; i < teamSize; i++) {
            printf("%d%s", team[i], (i == teamSize - 1) ? "" : " ");
        }
```

```
        return;
    }

    // Base case: reached the end of the employee list
    if (index == n) return;

    // Option 1: Include the current employee in the team
    team[teamSize] = skills[index];
    findTeam(index + 1, currentSum + skills[index], teamSize + 1);

    // Option 2: Exclude the current employee from the team
    if (!found) {
        findTeam(index + 1, currentSum, teamSize);
    }
}

int main() {
    // Read number of employees
    if (scanf("%d", &n) != 1) return 0;

    // Read skill contributions
    for (int i = 0; i < n; i++) {
        scanf("%d", &skills[i]);
    }

    // Read target skill value
    scanf("%d", &target);

    // Start search from the first employee
    findTeam(0, 0, 0);

    // If no subset was found after exploring all options
    if (!found) {
        printf("No valid team found");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Viyana is an event coordinator organizing multiple sessions for a tech conference. Each session has a start time and an end time.

Due to limited resources, Viyana wants to select a group of sessions such that:

No two selected sessions overlap in time.The total number of selected sessions is maximized.

However, to make the event more impactful, each session also has a priority score, and Viyana wants to maximize the total priority score instead of just the count.

Your task is to help Viyana select the optimal set of non-overlapping sessions that yields the maximum total priority score.

Input Format

The first line contains an integer n, representing the number of sessions.

The next n lines each contain three integers:start time, end time and priority score.

Output Format

The first line of output prints the maximum total priority score.

In the next line, print the selected sessions (indices starting from 1) in ascending order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
1 3 5

2 5 6
4 6 5
6 7 4
Output: 14
1 3 4

Answer

```
#include <stdio.h>
#include <stdbool.h>

typedef struct {
    int start, end, priority, id;
} Session;

int n;
Session sessions[20];
int bestScore = 0;
int bestCombination[20];
int bestCount = 0;

int currentCombination[20];

// Function to check if a session overlaps with any session already in the
selection
bool is_overlapping(int sessionIdx, int count) {
    for (int i = 0; i < count; i++) {
        int selectedIdx = currentCombination[i];
        // Two sessions [s1, e1] and [s2, e2] overlap if s1 < e2 and s2 < e1
        if (sessions[sessionIdx].start < sessions[selectedIdx].end &&
            sessions[selectedIdx].start < sessions[sessionIdx].end) {
            return true;
        }
    }
    return false;
}

void solve(int index, int currentScore, int count) {
    if (index == n) {
        if (currentScore > bestScore) {
            bestScore = currentScore;
            bestCount = count;
            for (int i = 0; i < count; i++) {
```

```
            bestCombination[i] = currentCombination[i];
        }
    }
    return;
}

// Option 1: Include the current session (if no overlap)
if (!is_overlapping(index, count)) {
    currentCombination[count] = index;
    solve(index + 1, currentScore + sessions[index].priority, count + 1);
}

// Option 2: Exclude the current session
solve(index + 1, currentScore, count);
}

int main() {
if (scanf("%d", &n) != 1) return 0;

for (int i = 0; i < n; i++) {
    scanf("%d %d %d", &sessions[i].start, &sessions[i].end, &sessions[i].priority);
    sessions[i].id = i + 1; // 1-based index
}

solve(0, 0, 0);

// Output the maximum priority score
printf("%d\n", bestScore);

// Output the session indices in ascending order
for (int i = 0; i < bestCount; i++) {
    printf("%d%s", sessions[bestCombination[i]].id, (i == bestCount - 1) ? "" : " ");
}

return 0;
}
```

Status : Correct **Marks : 10/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Raj is developing financial software that must identify all non-empty subsets of a given list of expenses whose total sum equals a specified target value.

Each expense can either be included or excluded from a subset. The same expense cannot be used more than once in a subset.

Your task is to print all such valid subsets. If no valid non-empty subset exists whose sum equals the target value, print:

No subset matches the target sum

This problem demonstrates how the number of possible subsets increases exponentially when using brute-force approaches.

Formula

Subset Sum = sum of selected elements in the subset

A subset is considered valid if:

sum(subset elements) = target

Input Format

The first line contains an integer n, representing the number of expenses.

The second line contains n space-separated integers representing the expense values.

The third line contains an integer t, representing the target sum.

Output Format

Print each valid non-empty subset whose elements sum to t, one subset per line.

Elements of a subset must be printed as space-separated integers.

If no such subset exists, print:

No subset matches the target sum

Refer to the sample output for formatting.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
1 2 3
3
Output: 1 2
3

Answer

```
#include <stdio.h>
#include <stdbool.h>

int n, target;
int expenses[20];
int subset[20];
bool found = false;

// Recursive function to find subsets
void findSubsets(int index, int currentSum, int subsetSize) {
    // Base Case: If we've considered all elements
    if (index == n) {
        // Check if sum matches target and subset is not empty
        if (currentSum == target && subsetSize > 0) {
            found = true;
            for (int i = 0; i < subsetSize; i++) {
                printf("%d%s", subset[i], (i == subsetSize - 1) ? "" : " ");
            }
            printf("\n");
        }
        return;
    }

    // Include the current expense in the subset
    subset[subsetSize] = expenses[index];
    findSubsets(index + 1, currentSum + expenses[index], subsetSize + 1);

    // Exclude the current expense from the subset
    findSubsets(index + 1, currentSum, subsetSize);
}

int main() {
    // Input number of expenses
    if (scanf("%d", &n) != 1) return 0;

    // Input the expense values
    for (int i = 0; i < n; i++) {
        scanf("%d", &expenses[i]);
    }

    // Input the target sum
```

```
    scanf("%d", &target);

    // Start backtracking from index 0
    findSubsets(0, 0, 0);

    // If no subset was ever printed, output the specific message
    if (!found) {
        printf("No subset matches the target sum");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Sanjana is a cybersecurity analyst responsible for securing communication in a corporate network. The network consists of multiple computers (nodes) connected by communication links (edges).

To ensure secure communication, Sanjana wants to select a group of computers such that every pair of selected computers is directly connected to each other. This ensures a fully trusted group where all members can communicate securely without intermediaries. Such a group is called a clique.

Sanjana is given a list of connections in the network, and she needs to determine whether there exists a clique of size k.

Input Format

The first line contains two integers n and m, representing the number of computers (nodes) and connections (edges).

The next m lines each contain two integers u and v, representing a connection between nodes u and v.

The last line contains an integer k, representing the required clique size.

Output Format

If a clique of size k exists, the output prints: "Clique Found: v1 v2 v3 ..."

Otherwise, the output prints: "No clique of given size exists"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 6
0 1
0 2
1 2
1 3
2 3
3 4
3
Output: Clique Found: 0 1 2

Answer

```
#include <stdio.h>
#include <stdbool.h>

int n, m, k;
bool adj[20][20];
int store[20];
bool found = false;

// Function to check if the current set of vertices in 'store' forms a clique
bool is_clique(int sz) {
    for (int i = 0; i < sz; i++) {
        for (int j = i + 1; j < sz; j++) {
            if (!adj[store[i]][store[j]]) {
                return false;
            }
        }
    }
    return true;
}

// Backtracking function to find a clique of size k
```

```
void findClique(int start, int sz) {
    if (found) return; // Stop recursion if we already found one

    // If we have collected k vertices, check if they form a clique
    if (sz == k) {
        if (is_clique(k)) {
            found = true;
            printf("Clique Found: ");
            for (int i = 0; i < k; i++) {
                printf("%d%s", store[i], (i == k - 1) ? "" : " ");
            }
        }
        return;
    }

    // Explore further vertices
    for (int i = start; i < n; i++) {
        store[sz] = i;
        findClique(i + 1, sz + 1);
        if (found) return;
    }
}

int main() {
    // Read n (nodes) and m (edges)
    if (scanf("%d %d", &n, &m) != 2) return 0;

    // Initialize adjacency matrix
    for (int i = 0; i < 20; i++) {
        for (int j = 0; j < 20; j++) {
            adj[i][j] = false;
        }
    }

    // Read edges
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = adj[v][u] = true;
    }

    // Read target clique size k
```

```
    scanf("%d", &k);

    // Start searching for combinations
    findClique(0, 0);

    if (!found) {
        printf("No clique of given size exists");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rehan, a logistics manager, is packing valuable items into two separate containers. Each item has a specific value, and he wants both containers to carry the same total value to ensure balance and fairness. He needs to decide if it's possible to divide the items this way and print one such valid division if it exists.

This problem is based on a classic NP-complete problem known as the Partition Problem, where finding a solution may require checking all combinations in the worst case.

Help him write a program to solve this problem.

Input Format

The first line of input consists of an integer n, representing the number of items.

The second line contains n space-separated integers, where each integer represents the value of an item.

Output Format

If a valid equal partition exists, print:

- Subset 1: x1 x2 x3 ...
- Subset 2: y1 y2 y3 ...

If no such partition exists, it prints "No valid equal partition exists."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
1 5 11 5
Output: Subset 1: 1 5 5
Subset 2: 11

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <numeric>

int n;
int items[20];
bool used[20]; // To track which items belong to Subset 1
bool found = false;

// Backtracking function to find a subset that sums to 'target'
void findPartition(int index, int currentSum, int target) {
    if (found) return;

    // Base case: if current sum equals the target, we found Subset 1
    if (currentSum == target) {
        found = true;

        printf("Subset 1: ");
        bool first = true;
        for (int i = 0; i < n; i++) {
            if (used[i]) {
                if (!first) printf(" ");
                printf("%d", items[i]);
                first = false;
            }
        }
        printf("\n");
```

```
        printf("Subset 2: ");
        first = true;
        for (int i = 0; i < n; i++) {
            if (!used[i]) {
                if (!first) printf(" ");
                printf("%d", items[i]);
                first = false;
            }
        }
        return;
    }

    if (index == n) return;

    // Option 1: Include items[index] in Subset 1
    used[index] = true;
    findPartition(index + 1, currentSum + items[index], target);

    // Option 2: Exclude items[index] (it will go to Subset 2)
    if (!found) {
        used[index] = false;
        findPartition(index + 1, currentSum, target);
    }
}

int main() {
    if (scanf("%d", &n) != 1) return 0;

    long long totalSum = 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &items[i]);
        totalSum += items[i];
        used[i] = false;
    }

    // If total sum is odd, it's impossible to divide into two equal integer subsets
    if (totalSum % 2 != 0) {
        printf("No valid equal partition exists.");
    } else {
        findPartition(0, 0, totalSum / 2);
        if (!found) {
```

```
            printf("No valid equal partition exists.");
        }
    }

    return 0;
}
```

Status : Wrong **Marks : 0/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 4

Section 1 : MCQ

1. How many conditions have to be met if an NP-complete problem is polynomially reducible?

Answer

1

Status : Wrong ***Marks : 0/1***

2. _____ is the class of decision problems that can be solved by non-deterministic polynomial algorithms.

Answer

P

Status : Wrong ***Marks : 0/1***

3. To which of the following classes does a CNF-satisfiability problem belong?

Answer

NP complete

Status : Correct ***Marks : 1/1***

4. For problems X and Y, Y is NP-complete and X reduces to Y in polynomial time. Which of the following is TRUE?

Answer

X is NP-complete

Status : Wrong ***Marks : 0/1***

5. A problem in NP is NP-complete if

Answer

Some problem in NP can be reduced to it in polynomial time

Status : Wrong ***Marks : 0/1***

6. Which class contains problems that can be solved in polynomial time using a deterministic machine?

Answer

P

Status : Correct ***Marks : 1/1***

7. What is the key property of problems in NP?

Answer

Their solutions cannot be verified

Status : Wrong ***Marks : 0/1***

8. What does it mean if a problem is in both NP and NP-Complete?

Answer

It is easier than P problems

Status : Wrong ***Marks : 0/1***

9. If P = NP is ever proven, what would it imply?

Answer

No problem in NP can be solved quickly

Status : Wrong ***Marks : 0/1***

10. What is the class NP named after?

Answer

Numerical Problems

Status : Wrong ***Marks : 0/1***

11. Which complexity class includes problems that are at least as hard as the hardest problems in NP?

Answer

P

Status : Wrong ***Marks : 0/1***

12. What is the defining property of NP-hard problems?

Answer

They are decision problems.

Status : Wrong ***Marks : 0/1***

13. In computational complexity theory, what does "NP" stand for?

Answer

Non-Deterministic Polynomial Time

Status : Correct ***Marks : 1/1***

14. What is the main characteristic that distinguishes NP-complete problems from NP-hard problems?

Answer

NP-complete problems are the hardest problems in NP.

Status : Correct ***Marks : 1/1***

15. The concept of "polynomial-time reduction" is commonly used to establish which of the following relationships between problems?

Answer

Status : Skipped ***Marks : 0/1***

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 42.5

Section 1 : Coding

1. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

Input Format

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

Output Format

Maximum matching after removing vertex

Sample Test Case

Input: 3
2
0 1
1 0
0 0
1
Output: Maximum matching after removing vertex 1: 1

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define MAX 20

int adj[MAX][MAX];
int match[MAX];
bool visited[MAX];
int m, n;

// DFS to find an augmenting path
bool can_match(int u, int n_right) {
   for (int v = 0; v < n_right; v++) {
      // If there is an edge and the vertex hasn't been visited in this path
      if (adj[u][v] && !visited[v]) {
         visited[v] = true;

         // If v is not matched OR the existing match for v can be matched
elsewhere
```

```
            if (match[v] < 0 || can_match(match[v], n_right)) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}

int main() {
    // Input sizes of the two partitions
    if (scanf("%d %d", &m, &n) != 2) return 0;

    // Input the adjacency matrix
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    int vertexToRemove;
    scanf("%d", &vertexToRemove);

    // Remove the specified vertex by clearing its row connections
    if (vertexToRemove >= 0 && vertexToRemove < m) {
        for (int j = 0; j < n; j++) {
            adj[vertexToRemove][j] = 0;
        }
    }

    int result = 0;
    // Initialize all right-partition nodes as unmatched (-1)
    for (int i = 0; i < n; i++) {
        match[i] = -1;
    }

    // Try to find a match for each node in the left partition
    for (int i = 0; i < m; i++) {
        memset(visited, false, sizeof(visited));
        if (can_match(i, n)) {
            result++;
        }
```

```
    }

    // Output formatted exactly as per sample requirements
    printf("Maximum matching after removing vertex %d: %d", vertexToRemove,
result);

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Charlie is working on optimizing the hiring process for interns at a tech startup. There are 4 open positions and 4 applicants, each having different skill sets.

Charlie wants to design a system that matches applicants to positions based on their suitability. The suitability is represented using a binary matrix, where:

1 indicates the applicant is suitable for the position.0 indicates the applicant is not suitable for the position.

Each applicant can be assigned to only one position, and each position can accept only one applicant.

The goal is to determine the maximum number of positions that can be filled by assigning suitable applicants.

Input Format

The input consists of 4 lines.

Each line contains 4 space-separated integers (0 or 1).

Each row represents an applicant.

Each column represents a job position.

Output Format

The output prints the result in the following format: "The job can hire up to X applicants" where X represents the maximum number of applicants that can be assigned to positions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 1 0 0
1 0 0 0
0 0 1 1
0 0 0 1
Output: The job can hire up to 4 applicants

Answer

```
#include <stdio.h>
#include <stdbool.h>

#define N 4

int adj[N][N];
int match[N];
bool visited[N];

// DFS to find an augmenting path for applicant u
bool can_assign(int u) {
    for (int v = 0; v < N; v++) {
        // If applicant is suitable for the job and job hasn't been visited in this path
        if (adj[u][v] && !visited[v]) {
            visited[v] = true;

            // If job v is not assigned OR the applicant already assigned to job v
            // can be moved to another job
            if (match[v] < 0 || can_assign(match[v])) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
```

```
}

int main() {
    // Read the 4x4 suitability matrix
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (scanf("%d", &adj[i][j]) != 1) return 0;
        }
    }

    // Initialize all jobs as unassigned (match = -1)
    for (int i = 0; i < N; i++) {
        match[i] = -1;
    }

    int result = 0;
    for (int i = 0; i < N; i++) {
        // Reset visited array for each applicant
        for (int j = 0; j < N; j++) {
            visited[j] = false;
        }

        // If we can find a matching for applicant i, increment count
        if (can_assign(i)) {
            result++;
        }
    }

    // Exact output format as per requirement
    printf("The job can hire up to %d applicants", result);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Emily is working on a project that involves managing relationships between two sets of entities, set U and set V. She wants to determine the maximum matching in a bipartite graph to optimize connections without

overlap.

Emily also needs the ability to remove a specific connection (edge) between two entities in order to analyze different scenarios and measure the impact on the size of the maximum matching.

Your task is to compute the size of the maximum matching before and after removing a given edge.

Note: Vertices in set U are numbered from 1 to n, and vertices in set V are numbered from 1 to m.

Input Format

The first line contains an integer n, representing the number of vertices in set U.

The second line contains an integer m, representing the number of vertices in set V.

The third line contains an integer e, representing the number of edges.

The next e lines each contain two integers u and v, representing an edge between vertex u (from set U) and vertex v (from set V).

The last line contains two integers u and v, representing the edge that must be removed.

Output Format

The first line displays: "Size of maximum matching before edge removal: X"

The second line displays: "Size of maximum matching after edge removal: Y"

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 3
3
3

0 1
0 2
1 2
1 2

Output: Size of maximum matching before edge removal: 1
Size of maximum matching after edge removal: 0

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define MAX 101

int adj[MAX][MAX];
int match[MAX];
bool visited[MAX];
int n, m, e;

// DFS to find an augmenting path
bool can_match(int u, int m_val) {
    for (int v = 1; v <= m_val; v++) {
        if (adj[u][v] && !visited[v]) {
            visited[v] = true;
            if (match[v] < 0 || can_match(match[v], m_val)) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}

// Function to calculate the maximum matching size
int get_max_matching() {
    int result = 0;
    memset(match, -1, sizeof(match));
    for (int i = 1; i <= n; i++) {
        memset(visited, false, sizeof(visited));
        if (can_match(i, m)) {
            result++;
        }
```

```
    }
    return result;
}

int main() {
    if (scanf("%d %d %d", &n, &m, &e) != 3) return 0;

    // Reset adjacency matrix
    for (int i = 0; i < MAX; i++) {
        for (int j = 0; j < MAX; j++) {
            adj[i][j] = 0;
        }
    }

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        // Note: Vertices are usually 1-indexed based on problem description
        // But sample input 1 uses 0, while sample 2 uses 1.
        // We handle both by adjusting indexing if necessary,
        // but adding 1 to 0-based inputs to fit 1-indexed logic is safest.
        adj[u + 1][v + 1] = 1;
    }

    int remU, remV;
    scanf("%d %d", &remU, &remV);

    // Calculate before removal
    int before = get_max_matching();

    // Remove the edge
    adj[remU + 1][remV + 1] = 0;

    // Calculate after removal
    int after = get_max_matching();

    printf("Size of maximum matching before edge removal: %d\n", before);
    printf("Size of maximum matching after edge removal: %d", after);

    return 0;
}
```

Status : Partially correct ***Marks : 2.5/10***

4. Problem Statement

In a computer lab, Alex is experimenting with task scheduling on a cluster of processors. The lab has 7 tasks that need to be executed and 5 processors available.

Each task can run only on certain processors due to compatibility constraints. These compatibilities are represented using a binary matrix, where:

1 indicates the task can run on that processor.0 indicates the task cannot run on that processor.

Each processor can execute only one task at a time, and each task can be assigned to only one processor. Alex wants to determine the maximum number of tasks that can be executed simultaneously.

Write a program to compute the maximum number of compatible task–processor assignments.

Input Format

The input consists of 7 lines.

Each line contains 5 space-separated integers (0 or 1).

Each row represents a task.

Each column represents a processor.

Output Format

The output prints the result in the format: "The processors can run a maximum of x tasks at one time." , where x is the maximum number of tasks that can be executed simultaneously.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 1 0 0 0
0 1 0 0 0
0 0 1 1 0
0 0 0 1 1
1 0 0 0 0
0 0 1 0 0
0 0 0 0 1

Output: The processors can run a maximum of 5 tasks at one time.

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define TASKS 7
#define PROCESSORS 5

int adj[TASKS][PROCESSORS];
int match[PROCESSORS];
bool visited[PROCESSORS];

// DFS to find an augmenting path for task u
bool can_assign(int u) {
   for (int v = 0; v < PROCESSORS; v++) {
      // If task u is compatible with processor v and v isn't visited in this path
      if (adj[u][v] && !visited[v]) {
         visited[v] = true;

         // If processor v is free OR the task currently assigned to v can be moved
         if (match[v] < 0 || can_assign(match[v])) {
            match[v] = u;
            return true;
         }
      }
   }
   return false;
}

int main() {
   // Read the 7x5 compatibility matrix
```

```
    for (int i = 0; i < TASKS; i++) {
        for (int j = 0; j < PROCESSORS; j++) {
            if (scanf("%d", &adj[i][j]) != 1) return 0;
        }
    }

    // Initialize all processors as unassigned (-1)
    for (int i = 0; i < PROCESSORS; i++) {
        match[i] = -1;
    }

    int total_tasks = 0;
    for (int i = 0; i < TASKS; i++) {
        // Reset visited array for each task's attempt
        memset(visited, false, sizeof(visited));

        if (can_assign(i)) {
            total_tasks++;
        }
    }

    // Output with exact formatting required by the problem
    printf("The processors can run a maximum of %d tasks at one time.",
total_tasks);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Alex is working on a project that involves pairing students from two different classes for a team project. The project's success depends on maximizing the number of compatible pairs between these two groups. Each student from one group can be paired with a student from the other group, but no student can belong to more than one pair.

Alex represents the compatibility between students using a binary adjacency matrix, where:

1 indicates that the students are compatible.0 indicates that the students are not compatible.

Write a program to determine the maximum number of compatible student pairs that can be formed.

Input Format

The first line contains two integers n and m, representing the number of students in the first group and second group, respectively.

The next n lines each contain m space-separated integers, representing the adjacency matrix of student compatibility.

Output Format

The output prints a single line in the following format: "Maximum Bipartite Matching: X" , where X is the maximum number of compatible student pairs.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 3
1 1 0
1 0 1
0 1 0
1 0 0
Output: Maximum Bipartite Matching: 3

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define MAX 105

int adj[MAX][MAX];
int match[MAX];
bool visited[MAX];
int n, m;
```

```
// Standard DFS to find an augmenting path for student u
bool can_match(int u) {
    for (int v = 0; v < m; v++) {
        // If student u is compatible with student v and v hasn't been visited in this
search
        if (adj[u][v] && !visited[v]) {
            visited[v] = true;

            // If student v is currently unmatched OR their current partner
            // can be reassigned to someone else
            if (match[v] < 0 || can_match(match[v])) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}

int main() {
    // Read the number of students in both groups
    if (scanf("%d %d", &n, &m) != 2) return 0;

    // Read the compatibility matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    // Initialize all students in the second group as unmatched
    for (int i = 0; i < m; i++) {
        match[i] = -1;
    }

    int result = 0;
    for (int i = 0; i < n; i++) {
        // Reset visited status for each student in the first group
        memset(visited, false, sizeof(visited));

        // If an augmenting path is found, increment the matching count
```

```
        if (can_match(i)) {
            result++;
        }
    }

    // Exact output format as per requirement
    printf("Maximum Bipartite Matching: %d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 42.5

Section 1 : Coding

1. Problem Statement

You are given a bipartite graph represented using an edge list. The graph contains two sets of nodes tasks and processors. Each edge connects a task node to a processor node, indicating that the processor can execute that task.

Your goal is to determine the maximum matching in the bipartite graph. A matching is a set of edges such that:

Each task is assigned to at most one processor.Each processor is assigned to at most one task.

Write a program to compute the maximum number of task–processor assignments possible.

Input Format

The first line contains an integer T, representing the number of tasks.

The second line contains an integer P, representing the number of processors.

The third line contains an integer E, representing the number of edges in the bipartite graph.

The next E lines each contain two space-separated integers task and processor where this indicates that the given task can be assigned to the specified processor.

Output Format

The output prints the result in the following format: "The maximum matching is: X", where X is the maximum number of matches that can be made in the bipartite graph.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3
5
1 1
2 2
4 3
3 3
4 1
Output: The maximum matching is: 3

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 20

int graph[MAX][MAX];   // adjacency matrix
int matchR[MAX];       // processor -> task
int visited[MAX];
```

```
int T, P;

// DFS function to find matching
int bpm(int task) {
    for (int proc = 1; proc <= P; proc++) {
        if (graph[task][proc] && !visited[proc]) {
            visited[proc] = 1;

            // If processor is not assigned OR can reassign
            if (matchR[proc] == -1 || bpm(matchR[proc])) {
                matchR[proc] = task;
                return 1;
            }
        }
    }
    return 0;
}

int maxMatching() {
    int result = 0;

    // Initialize all processors as free
    for (int i = 1; i <= P; i++)
        matchR[i] = -1;

    // Try to match each task
    for (int task = 1; task <= T; task++) {
        memset(visited, 0, sizeof(visited));

        if (bpm(task))
            result++;
    }

    return result;
}

int main() {
    int E;
    scanf("%d", &T);
    scanf("%d", &P);
    scanf("%d", &E);
```

```
    // Initialize graph
    memset(graph, 0, sizeof(graph));

    for (int i = 0; i < E; i++) {
        int t, p;
        scanf("%d %d", &t, &p);
        graph[t][p] = 1;
    }

    int maxMatch = maxMatching();
    printf("The maximum matching is: %d", maxMatch);

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Irfan is working on a project that involves managing relationships between two sets of entities, set U and set V. He wants to determine the maximum matching in a bipartite graph to optimize connections without overlap.Irfan also wants to explore different scenarios by adding a new connection (edge) between two entities and observing how it affects the maximum matching.

Your task is to compute the size of the maximum matching before and after adding a specified edge.

Note: Vertices in set U are numbered from 0 to n-1, and vertices in set V are numbered from 1 to m.

Input Format

The first line contains an integer n, representing the number of vertices in set U.

The second line contains an integer m, representing the number of vertices in set V.

The third line contains an integer e, representing the number of edges.

The next e lines each contain two integers u and v, representing an edge between vertex u from set U and vertex v from set V.

The last line contains two integers u and v, representing the edge to be added.

Output Format

The output prints the results in the following format:

Size of maximum matching before adding edges: X

Size of maximum matching after adding edges: Y

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
4
3
0 1
0 2
1 1
1 2
Output: Size of maximum matching before adding edges: 1
Size of maximum matching after adding edges: 1

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 105

int graph[MAX][MAX];
int matchR[MAX];
int visited[MAX];
int n, m;

// DFS for matching
int bpm(int u) {
```

```
    for (int v = 1; v <= m; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (matchR[v] == -1 || bpm(matchR[v])) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

// Find maximum matching
int maxMatching() {
    int result = 0;

    for (int i = 1; i <= m; i++)
        matchR[i] = -1;

    for (int u = 0; u < n; u++) {
        memset(visited, 0, sizeof(visited));
        if (bpm(u))
            result++;
    }

    return result;
}

int main() {
    int e;
    scanf("%d", &n);
    scanf("%d", &m);
    scanf("%d", &e);

    memset(graph, 0, sizeof(graph));

    // Read edges
    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
```

```
    }

    // Compute before adding edge
    int before = maxMatching();

    // Read new edge
    int u, v;
    scanf("%d %d", &u, &v);
    graph[u][v] = 1;

    // Compute after adding edge
    int after = maxMatching();

    printf("Size of maximum matching before adding edges: %d\n", before);
    printf("Size of maximum matching after adding edges: %d", after);

    return 0;
}
```

Status : Partially correct **Marks : 2.5/10**

3. Problem Statement

Jack wants to create a program to manage a bipartite graph, which contains two sets of vertices.He needs the program to:

Initialize the graph with a specific size,Add edges between the two sets of vertices, andFind pairings where each vertex from the first set is matched with at most one vertex from the second set, and no two vertices from the first set are matched with the same vertex from the second set.

The goal is to determine the maximum matching pairs in the bipartite graph.

Input Format

The first line contains an integer m, representing the number of vertices in the first set.

The second line contains an integer n, representing the number of vertices in the second set.

The third line contains an integer e, representing the number of edges in the graph.

The next e lines each contain two integers u and v, representing an edge between vertex u in the first set and vertex v in the second set.

Output Format

The output prints the maximum matching pairs in increasing order of u, where each pair is printed in the format: (u, v)

If there are no matching pairs, print: "No matching found"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3
5
1 1
2 2
4 3
3 3
4 1
Output: (1, 1)
(2, 2)
(3, 3)

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 20

int graph[MAX][MAX];
int matchR[MAX];
int visited[MAX];
int m, n;

// DFS for matching
```

```
int bpm(int u) {
    for (int v = 1; v <= n; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (matchR[v] == -1 || bpm(matchR[v])) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

// Find maximum matching
void maxMatching() {
    for (int i = 1; i <= n; i++)
        matchR[i] = -1;

    for (int u = 1; u <= m; u++) {
        memset(visited, 0, sizeof(visited));
        bpm(u);
    }
}

int main() {
    int e;
    scanf("%d", &m);
    scanf("%d", &n);
    scanf("%d", &e);

    memset(graph, 0, sizeof(graph));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    maxMatching();

    // Store result pairs
```

```
    int found = 0;
    int resultU[MAX], resultV[MAX];
    int count = 0;

    for (int v = 1; v <= n; v++) {
        if (matchR[v] != -1) {
            resultU[count] = matchR[v];
            resultV[count] = v;
            count++;
            found = 1;
        }
    }

    // Sort pairs by u (simple bubble sort since small size)
    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
            if (resultU[i] > resultU[j]) {
                int tempU = resultU[i];
                int tempV = resultV[i];
                resultU[i] = resultU[j];
                resultV[i] = resultV[j];
                resultU[j] = tempU;
                resultV[j] = tempV;
            }
        }
    }

    if (!found) {
        printf("No matching found");
    } else {
        for (int i = 0; i < count; i++) {
            printf("(%d, %d)", resultU[i], resultV[i]);
            if (i != count - 1)
                printf("\n");
        }
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Alex needs to pair students from two different classes for a project, aiming to maximize the number of compatible pairs. Each student can be paired with only one student from the other class. The compatibility between students is represented as a binary adjacency matrix.

Alex wants to write a program to find the maximum number of compatible student pairs that can be formed.

Input Format

The first line contains an integer n, representing the number of students in each group.

The second line contains an integer m, representing the number of students in the second group.

The next n lines contain m space-separated integers each, representing the adjacency matrix of student compatibility.

Output Format

The single line containing the phrase "Maximum Bipartite Matching: x" where x is an integer representing the maximum number of compatible pairs that can be formed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3
1 1 0
1 0 1
0 1 0
1 0 0

Output: Maximum Bipartite Matching: 3

Answer

```
#include <iostream>
#include <vector>
using namespace std;

int n, m;
vector<vector<int>> graph;
vector<int> match;
vector<int> vis;

bool dfs(int u) {
    for (int v = 0; v < m; v++) {
        if (graph[u][v] && !vis[v]) {
            vis[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}

int maxBipartiteMatching() {
    match.assign(m, -1);
    int result = 0;

    for (int i = 0; i < n; i++) {
        vis.assign(m, 0);
        if (dfs(i))
            result++;
    }
    return result;
}

int main() {
    cin >> n >> m;

    graph.assign(n, vector<int>(m));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
```

```
            cin >> graph[i][j];
        }
    }

    int ans = maxBipartiteMatching();

    cout << "Maximum Bipartite Matching: " << ans;

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

Input Format

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

Output Format

Maximum matching after removing vertex

Sample Test Case

Input: 3
2
0 1
1 0
0 0
1
Output: Maximum matching after removing vertex 1: 1

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define MAX 20

int adj[MAX][MAX];
int match[MAX];
bool visited[MAX];
int m, n;

// Standard DFS to find an augmenting path for bipartite matching
bool can_match(int u, int n_right) {
    for (int v = 0; v < n_right; v++) {
        if (adj[u][v] && !visited[v]) {
            visited[v] = true;
            if (match[v] < 0 || can_match(match[v], n_right)) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}

int main() {
    // Input number of vertices in both partitions
    if (scanf("%d %d", &m, &n) != 2) return 0;

    // Input the adjacency matrix
```

```
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    int vertexToRemove;
    scanf("%d", &vertexToRemove);

    // "Remove" the vertex by clearing its row in the adjacency matrix
    // Note: If vertex indices are 0-based, we use vertexToRemove directly.
    // If 1-based, we would use vertexToRemove - 1.
    // Sample indicates 0-based or specific index logic.
    if (vertexToRemove >= 0 && vertexToRemove < m) {
        for (int j = 0; j < n; j++) {
            adj[vertexToRemove][j] = 0;
        }
    }

    int result = 0;
    // Initialize matches as -1 (unmatched)
    memset(match, -1, sizeof(match));

    // Calculate maximum matching
    for (int i = 0; i < m; i++) {
        memset(visited, false, sizeof(visited));
        if (can_match(i, n)) {
            result++;
        }
    }

    printf("Maximum matching after removing vertex %d: %d", vertexToRemove,
result);

    return 0;
}
```

Status : Correct **Marks : 10/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Alice needs to assign works to employees in a company. Each employee can be assigned at most one work, and each work can be assigned to at most one employee. Her goal is to maximize the number of employees who are assigned works, ensuring that each work is assigned to exactly one employee and each employee is assigned at most one work.

Help Alice to complete the task.

Input Format

The first line consists of an integer N, representing the number of connections between employees and works.

The next N lines consists of two space-separated integers X and Y, representing a connection where X is the index of an employee (0-based indexing), and Y is

the index of a work (0-based indexing), indicating that employee X can perform work Y.

Output Format

The first line of the output displays "Maximum number of employees that could get works are: a", where a is an integer representing the maximum number of employees that could be assigned works according to the given conditions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9
0 0
0 1
0 2
1 0
2 1
3 4
4 3
4 4
5 5
Output: Maximum number of employees that could get works are: 6

Answer

```
import java.util.*;

public class Main {

   static boolean bpm(int u, boolean[] seen, int[] matchR, List<List<Integer>> adj)
{
      for (int v : adj.get(u)) {
         if (!seen[v]) {
            seen[v] = true;

            // If work not assigned OR can reassign
            if (matchR[v] == -1 || bpm(matchR[v], seen, matchR, adj)) {
               matchR[v] = u;
               return true;
            }
```

```
        }
    }
    return false;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int N = sc.nextInt();

    List<List<Integer>> adj = new ArrayList<>();

    // Since max index ≤ N, create N+1 lists
    for (int i = 0; i <= N; i++) {
        adj.add(new ArrayList<>());
    }

    int maxWork = 0;

    for (int i = 0; i < N; i++) {
        int emp = sc.nextInt();
        int work = sc.nextInt();

        adj.get(emp).add(work);
        maxWork = Math.max(maxWork, work);
    }

    int[] matchR = new int[maxWork + 1];
    Arrays.fill(matchR, -1);

    int result = 0;

    // Try assigning each employee
    for (int i = 0; i <= N; i++) {
        boolean[] seen = new boolean[maxWork + 1];

        if (bpm(i, seen, matchR, adj)) {
            result++;
        }
    }

    System.out.println("Maximum number of employees that could get works
```

```
are: " + result);
    }
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Given an undirected bipartite graph with X vertices on the left side and Y vertices on the right side, the graph may contain multiple edges between the same pair of vertices.

Your task is to determine the minimum vertex cover for this graph.

A vertex cover of a graph is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

Input Format

The first line contains two space-separated integers X and Y, representing the number of vertices in the left set and right set, respectively.

The second line contains an integer e, representing the number of edges in the bipartite graph.

The next e lines each contain two space-separated integers u and v, representing an edge between: vertex u in the left set and vertex v in the right set

Output Format

The output prints the vertices in the minimum vertex cover, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 4
4

0 1
0 2
1 3
2 3

Output: 0 3

Answer

```
import java.util.*;

public class Main {

   static boolean bpm(int u, boolean[] seen, int[] matchR, List<List<Integer>> adj)
{
      for (int v : adj.get(u)) {
         if (!seen[v]) {
            seen[v] = true;

            if (matchR[v] == -1 || bpm(matchR[v], seen, matchR, adj)) {
               matchR[v] = u;
               return true;
            }
         }
      }
      return false;
   }

   static void dfs(int u, boolean[] visL, boolean[] visR, int[] matchR,
List<List<Integer>> adj) {
      visL[u] = true;

      for (int v : adj.get(u)) {
         if (!visR[v]) {
            visR[v] = true;

            if (matchR[v] != -1 && !visL[matchR[v]]) {
               dfs(matchR[v], visL, visR, matchR, adj);
            }
         }
      }
   }

   public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);

int X = sc.nextInt();
int Y = sc.nextInt();

int e = sc.nextInt();

List<List<Integer>> adj = new ArrayList<>();
for (int i = 0; i < X; i++) {
    adj.add(new ArrayList<>());
}

for (int i = 0; i < e; i++) {
    int u = sc.nextInt();
    int v = sc.nextInt();
    adj.get(u).add(v);
}

int[] matchR = new int[Y];
Arrays.fill(matchR, -1);

// Step 1: Maximum Matching
for (int i = 0; i < X; i++) {
    boolean[] seen = new boolean[Y];
    bpm(i, seen, matchR, adj);
}

// Step 2: Find unmatched left vertices
boolean[] visL = new boolean[X];
boolean[] visR = new boolean[Y];

boolean[] matchedLeft = new boolean[X];
for (int v = 0; v < Y; v++) {
    if (matchR[v] != -1) {
        matchedLeft[matchR[v]] = true;
    }
}

// DFS from unmatched left vertices
for (int i = 0; i < X; i++) {
    if (!matchedLeft[i]) {
        dfs(i, visL, visR, matchR, adj);
```

```
            }
        }

        // Step 3: Print result
        // Left NOT visited + Right visited
        for (int i = 0; i < X; i++) {
            if (!visL[i]) {
                System.out.print(i + " ");
            }
        }

        for (int i = 0; i < Y; i++) {
            if (visR[i]) {
                System.out.print(i + " ");
            }
        }
    }
}
```

Status : Wrong ***Marks : 0/10***

3. Problem Statement

You are given an undirected graph with 'num_vertices' vertices and 'num_edges' edges. Your task is to determine whether the given graph is bipartite or not. A graph is bipartite if it can be divided into two disjoint sets of vertices such that each edge connects a vertex from one set to a vertex from the other set.

Input Format

The first line contains two integers separated by a space: 'num_vertices' and 'num_edges'. These represent the number of vertices and edges in the graph, respectively.

The next 'num_edges' lines each contain two integers 'u' and 'v' separated by a space, representing an edge between vertex 'u' and vertex 'v'.

Output Format

The output displays a single line indicating whether the given graph is bipartite or not.

If the graph is bipartite, print "The graph is bipartite."

If the graph is not bipartite, print "The graph is not bipartite."

Sample Test Case

Input: 4 4
0 1
1 2
2 3
3 0
Output: The graph is bipartite.

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int adj[MAX][MAX];
int color[MAX]; // -1 = uncolored, 0 and 1 are two colors

int isBipartite(int V) {
   int queue[MAX], front, rear;

   for (int i = 0; i < V; i++) {
      if (color[i] == -1) {
         front = rear = 0;

         queue[rear++] = i;
         color[i] = 0;

         while (front < rear) {
            int u = queue[front++];

            for (int v = 0; v < V; v++) {
               if (adj[u][v]) {
                  // If not colored
                  if (color[v] == -1) {
                     color[v] = 1 - color[u];
                     queue[rear++] = v;
                  }
```

```
                // If same color, not bipartite
                else if (color[v] == color[u]) {
                    return 0;
                }
            }
        }
    }
}
    return 1;
}

int main() {
    int V, E;
    scanf("%d %d", &V, &E);

    // Initialize adjacency matrix
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            adj[i][j] = 0;

    // Initialize colors
    for (int i = 0; i < V; i++)
        color[i] = -1;

    // Input edges
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    if (isBipartite(V))
        printf("The graph is bipartite.");
    else
        printf("The graph is not bipartite.");

    return 0;
}
```

Status : Correct *Marks : 10/10*

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 7

Section 1 : Coding

1. _____ is a matching with the largest number of edges.

Answer

Maximum bipartite matching

Status : Correct ***Marks : 1/1***

2. How many colors are used in a bipartite graph?

Answer

2

Status : Correct ***Marks : 1/1***

3. What is the simplest method to prove that a graph is bipartite?

Answer

It does not have a cycle of an odd length

Status : Correct ***Marks : 1/1***

4. A matching that matches all the vertices of a graph is called?

Answer

Perfect matching

Status : Correct ***Marks : 1/1***

5. In a bipartite graph G=(V,U,E), the matching of a free vertex in V to a free vertex in U called?

Answer

Weight matching

Status : Wrong ***Marks : 0/1***

6. There are four students in a class namely A, B, C and D. A tells that a triangle is a bipartite graph. B tells pentagon is a bipartite graph. C tells square is a bipartite graph. D tells heptagon is a bipartite graph. Who among the following is correct?

Answer

B

Status : Wrong ***Marks : 0/1***

7. Which of the following is not a property of the bipartite graph?

Answer

Symmetric Spectrum

Status : Wrong ***Marks : 0/1***

8. The maximum number of edges in a bipartite graph with 14 vertices is _____.

Answer

49

Status : Correct ***Marks : 1/1***

9. What does the Halting Problem attempt to determine?

Answer

Whether a program will produce the correct output

Status : Wrong ***Marks : 0/1***

10. The Halting Problem was introduced by which computer scientist?

Answer

Edsger W. Dijkstra

Status : Wrong ***Marks : 0/1***

11. Why is the Halting Problem considered unsolvable?

Answer

Because programming languages are too complex

Status : Wrong ***Marks : 0/1***

12. In computation theory, what does an undecidable problem mean?

Answer

A problem that cannot be solved by any algorithm for all inputs

Status : Correct ***Marks : 1/1***

13. In the context of the Halting Problem, what does the term halting refer to?

Answer

A program generating an error

Status : Wrong ***Marks : 0/1***

14. What is the main implication of the Halting Problem for computer science?

Answer

Programs cannot contain loops

Status : Wrong ***Marks : 0/1***

15. The Halting Problem generally takes which of the following as input?

Answer

A program and its input data

Status : Correct ***Marks : 1/1***

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 32

Section 1 : Coding

1. Problem Statement

Meera, the director of a national disaster response team, must deploy three specialized rescue units to three critical disaster sites. Each rescue unit has a different response time (in minutes) to reach each site due to distance, terrain, and accessibility constraints.

However, Meera has an additional operational rule:

Rescue Unit 1 cannot be assigned to Site 3 due to equipment limitations.Rescue Unit 2 must not be assigned to Site 1 because of route restrictions.

Meera decides to use the Branch and Bound technique to explore only valid and efficient assignments while respecting all constraints.

Your task is to determine the total response time of the best valid

deployment plan that satisfies all given restrictions.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3 × 3 response time matrix.

Each cell (i, j) represents the response time (in minutes) if the ith rescue unit is deployed to the jth disaster site.

Output Format

The output should display the total response time in the following exact format: "Total Response Time: X" ,where X is the total response time of the best valid deployment plan.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 12 25 40
30 14 28
16 18 11
Output: Total Response Time: 37

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define N 3

int cost[N][N];
bool assigned[N]; // tracks assigned sites
int minCost = INT_MAX;

// Branch and Bound function
void solve(int unit, int currCost) {
    // If all units assigned
    if (unit == N) {
```

```
        if (currCost < minCost)
            minCost = currCost;
        return;
    }

    for (int site = 0; site < N; site++) {

        // Check if site already assigned
        if (!assigned[site]) {

            // Apply constraints
            if ((unit == 0 && site == 2) ||  // Unit 1 -> Site 3 not allowed
                (unit == 1 && site == 0))    // Unit 2 -> Site 1 not allowed
                continue;

            int newCost = currCost + cost[unit][site];

            // Pruning
            if (newCost < minCost) {
                assigned[site] = true;

                solve(unit + 1, newCost);

                assigned[site] = false; // backtrack
            }
        }
    }
}

int main() {
    // Input matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &cost[i][j]);

    // Initialize
    for (int i = 0; i < N; i++)
        assigned[i] = false;

    solve(0, 0);

    printf("Total Response Time: %d", minCost);
```

```
    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W. Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i-th integer represents the weight of the i-th item.

The third line contains n integers, where the i-th integer represents the value of the i-th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180

Output: 280

Answer

```
#include <stdio.h>

int max(int a, int b) {
   return (a > b) ? a : b;
}

int main() {
   int n, W;
   scanf("%d %d", &n, &W);

   int wt[n], val[n];

   for (int i = 0; i < n; i++)
      scanf("%d", &wt[i]);

   for (int i = 0; i < n; i++)
      scanf("%d", &val[i]);

   int dp[n+1][W+1];

   // Initialize
   for (int i = 0; i <= n; i++) {
      for (int w = 0; w <= W; w++) {
         if (i == 0 || w == 0)
            dp[i][w] = 0;
         else if (wt[i-1] <= w)
            dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]],
                           dp[i-1][w]);
         else
            dp[i][w] = dp[i-1][w];
      }
   }

   printf("%d", dp[n][W]);
```

```
    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Commander Vikram is preparing a spacecraft for an interplanetary research mission. The spacecraft has a limited cargo capacity, so he must carefully decide which scientific equipment modules to load.

Each module has a scientific importance value and a specific weight. Commander Vikram wants to load the spacecraft in such a way that the total scientific value of the selected modules is maximized without exceeding the cargo weight limit.

To solve this efficiently, he uses the Branch and Bound technique applied to the 0/1 Knapsack problem, which explores possible combinations of equipment and eliminates options that cannot lead to better solutions.

Your task is to help Commander Vikram determine the maximum achievable value and the selected module weights that should be loaded into the spacecraft.

Note: The weights must be printed in reverse order of the input, meaning the algorithm starts evaluating items from the last item and prints the selected weights first.

Input Format

The first line contains an integer n, representing the number of equipment modules available.

The second line contains n space-separated integers val[i], representing the scientific value of each module.

The third line contains n space-separated integers wt[i], representing the weight of each module.

The fourth line contains an integer W, representing the maximum cargo capacity of the spacecraft.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, printed in reverse order of the input.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3
60 100 120
10 20 30
50

Output: Maximum value: 220
Selected weights: 30 20

Answer

```
#include <stdio.h>

int n, W;
int val[20], wt[20];

int maxValue = 0;
int bestSet[20];
int currSet[20];

void knapsack(int i, int currWeight, int currValue) {
    // If weight exceeds, stop
    if (currWeight > W)
        return;

    // If all items considered
    if (i == n) {
        if (currValue > maxValue) {
            maxValue = currValue;
```

```
            for (int j = 0; j < n; j++)
                bestSet[j] = currSet[j];
        }
        return;
    }

    // Include item
    currSet[i] = 1;
    knapsack(i + 1, currWeight + wt[i], currValue + val[i]);

    // Exclude item
    currSet[i] = 0;
    knapsack(i + 1, currWeight, currValue);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);

    knapsack(0, 0, 0);

    printf("Maximum value: %d\n", maxValue);
    printf("Selected weights: ");

    // Print in reverse order
    for (int i = n - 1; i >= 0; i--) {
        if (bestSet[i] == 1)
            printf("%d ", wt[i]);
    }

    return 0;
}
```

Status : Partially correct ***Marks : 2/10***

4. Problem Statement

You are tasked with solving the Travelling Salesman Problem (TSP) for a given set of cities.

Given the distances between pairs of cities, your goal is to find the shortest route that visits each city exactly once and returns to the starting city.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input contains an integer n, representing the number of cities.

The following n lines each contain n space-separated integers, representing the distances between pairs of cities.

The distance between city i and city j is given at the ith row and jth column of the matrix.

Output Format

The first line displays "The Path is: " followed by the sequence of cities visited in the shortest path,

separated by "--->".

The second line displays "Minimum cost is X", where X is the minimum cost of the shortest path found.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 4 1 3
4 0 2 1
1 2 0 5
3 1 5 0
Output: The Path is: 1--->3--->2--->4--->1

Minimum cost is 7

Answer

```
#include <stdio.h>
#include <limits.h>

int n;
int cost[10][10];
int visited[10];
int path[10], bestPath[10];
int minCost = INT_MAX;

void tsp(int city, int count, int currCost) {
   // If all cities visited and return to start
   if (count == n && cost[city][0] != 0) {
      int totalCost = currCost + cost[city][0];
      if (totalCost < minCost) {
         minCost = totalCost;
         for (int i = 0; i < n; i++)
            bestPath[i] = path[i];
      }
      return;
   }

   for (int i = 0; i < n; i++) {
      if (!visited[i] && cost[city][i] != 0) {
         int newCost = currCost + cost[city][i];

         // Branch & Bound (Pruning)
         if (newCost < minCost) {
            visited[i] = 1;
            path[count] = i;

            tsp(i, count + 1, newCost);

            visited[i] = 0; // backtrack
         }
      }
   }
}

int main() {
```

```
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    // Initialize
    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    path[0] = 0;

    tsp(0, 1, 0);

    // Output
    printf("The Path is: ");
    for (int i = 0; i < n;
```

Status : Wrong **Marks : 0/10**

5. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n, the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The ith integer on the jth line represents the cost of traveling from location i to location j.

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 10 8 9 7
10 0 10 5 6
8 10 0 8 9
9 5 8 0 6
7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1
Minimum cost is 34

Answer

```
import java.util.*;

public class Main {
    static int n;
    static int[][] cost;
    static boolean[] visited;
    static int[] path, bestPath;
    static int minCost = Integer.MAX_VALUE;

    static void tsp(int city, int count, int currCost) {
        // If all cities visited and return to start
        if (count == n && cost[city][0] > 0) {
            int totalCost = currCost + cost[city][0];

            if (totalCost < minCost) {
                minCost = totalCost;
                for (int i = 0; i < n; i++) {
                    bestPath[i] = path[i];
```

```
            }
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[city][i] > 0) {
            int newCost = currCost + cost[city][i];

            // Branch & Bound (Pruning)
            if (newCost < minCost) {
                visited[i] = true;
                path[count] = i;

                tsp(i, count + 1, newCost);

                visited[i] = false; // backtrack
            }
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    n = sc.nextInt();
    cost = new int[n][n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cost[i][j] = sc.nextInt();

    visited = new boolean[n];
    path = new int[n];
    bestPath = new int[n];

    visited[0] = true;
    path[0] = 0;

    tsp(0, 1, 0);

    // Output
```

```
        System.out.print("Shortest Path: ");
        for (int i = 0; i < n; i++) {
            System.out.print((bestPath[i] + 1) + " ");
        }
        System.out.println("1");

        System.out.println("Minimum cost is " + minCost);
    }
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 48.5

Section 1 : Coding

1. Problem Statement

Arun, the operations manager of a logistics company, needs to assign three delivery drivers to three delivery zones.

Each driver requires a different amount of fuel (in liters) to complete deliveries in each zone due to varying distances and traffic conditions. Arun wants to assign exactly one driver to each zone such that the total fuel consumption is minimized.

To efficiently explore possible assignments and avoid unnecessary computations, Arun decides to use the Branch and Bound technique.

Your task is to help Arun determine the optimal assignment of drivers to zones that results in the least total fuel consumption.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3 × 3 matrix.

Each cell (i, j) represents the fuel required (in liters) if the ith driver is assigned to the jth zone.

Output Format

The output should display the optimal assignment in the following format:

Optimal Assignment:

Driver 1 -> Zone X

Driver 2 -> Zone Y

Driver 3 -> Zone Z

Where:

Each driver is assigned to exactly one zone.

Each zone is assigned to exactly one driver.

The assignment must produce the least total fuel consumption.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8 6 10
9 7 4
3 12 5
Output: Optimal Assignment:
Driver 1 -> Zone 2
Driver 2 -> Zone 3
Driver 3 -> Zone 1

Answer

```c
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define N 3

int cost[N][N];
bool assigned[N];   // track zones
int bestAssign[N];  // store best assignment
int tempAssign[N];  // current assignment
int minCost = INT_MAX;

// Branch and Bound function
void solve(int driver, int currentCost) {
    // If all drivers assigned
    if (driver == N) {
        if (currentCost < minCost) {
            minCost = currentCost;
            for (int i = 0; i < N; i++)
                bestAssign[i] = tempAssign[i];
        }
        return;
    }

    // Pruning
    if (currentCost >= minCost)
        return;

    // Try assigning zones
    for (int j = 0; j < N; j++) {
        if (!assigned[j]) {
            assigned[j] = true;
            tempAssign[driver] = j;

            solve(driver + 1, currentCost + cost[driver][j]);

            assigned[j] = false; // backtrack
        }
    }
}

int main() {
```

```
    // Input cost matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &cost[i][j]);

    // Initialize
    for (int i = 0; i < N; i++)
        assigned[i] = false;

    solve(0, 0);

    // Print result
    printf("Optimal Assignment:\n");
    for (int i = 0; i < N; i++) {
        printf("Driver %d -> Zone %d\n", i + 1, bestAssign[i] + 1);
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Captain Arvind has discovered a hidden island filled with valuable treasures. Before leaving the island, he must load the treasures onto his small ship. However, the ship has a strict weight capacity, so he cannot carry everything he finds.

Each treasure chest has a specific value and weight. Captain Arvind wants to select the best combination of treasure chests to load onto the ship so that the total value of the treasures is maximized without exceeding the ship's weight limit.

To solve this efficiently, Captain Arvind decides to use the Branch and Bound technique applied to the 0/1 Knapsack problem, which systematically explores possible combinations and eliminates choices that cannot produce better results.

Your task is to help Captain Arvind determine the maximum total treasure

value he can carry on his ship.

Input Format

The first line contains an integer n, representing the total number of treasure chests.

The next n lines contain two space-separated integers: val[i] w[i]

where

- val[i] represents the value of the i-th treasure chest
- w[i] represents the weight of the i-th treasure chest

The last line contains an integer W, representing the maximum weight capacity of the ship.

Output Format

The output prints "Optimal Total Value: X" , where X is the maximum value that Captain Arvind can obtain using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
60 10
100 20
120 30
50
Output: Optimal Total Value: 220

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

typedef struct {
```

```
    int value, weight;
    float ratio;
} Item;

typedef struct {
    int level, profit, weight;
    float bound;
} Node;

Item items[MAX];
int n, W;

// Sort by value/weight ratio (descending)
int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    else return -1;
}

// Calculate upper bound
float bound(Node u) {
    if (u.weight >= W)
        return 0;

    float profit_bound = u.profit;
    int j = u.level + 1;
    int totalWeight = u.weight;

    // Take full items
    while (j < n && totalWeight + items[j].weight <= W) {
        totalWeight += items[j].weight;
        profit_bound += items[j].value;
        j++;
    }

    // Take fraction of next item
    if (j < n)
        profit_bound += (W - totalWeight) * items[j].ratio;

    return profit_bound;
}
```

```
// Branch and Bound Knapsack
int knapsack() {
    Node u, v;
    Node queue[1000];
    int front = 0, rear = 0;

    int maxProfit = 0;

    // Start node
    u.level = -1;
    u.profit = 0;
    u.weight = 0;
    u.bound = bound(u);

    queue[rear++] = u;

    while (front < rear) {
        u = queue[front++];

        if (u.level == n - 1)
            continue;

        // Next level
        v.level = u.level + 1;

        // Include item
        v.weight = u.weight + items[v.level].weight;
        v.profit = u.profit + items[v.level].value;

        if (v.weight <= W && v.profit > maxProfit)
            maxProfit = v.profit;

        v.bound = bound(v);

        if (v.bound > maxProfit)
            queue[rear++] = v;

        // Exclude item
        v.weight = u.weight;
        v.profit = u.profit;
        v.bound = bound(v);
```

```
        if (v.bound > maxProfit)
            queue[rear++] = v;
    }

    return maxProfit;
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &items[i].value, &items[i].weight);
        items[i].ratio = (float)items[i].value / items[i].weight;
    }

    scanf("%d", &W);

    // Sort items
    qsort(items, n, sizeof(Item), cmp);

    int result = knapsack();

    printf("Optimal Total Value: %d", result);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Dr. Aarav is a field scientist preparing for an important research expedition to a remote mountain region. Since the research base is located in a difficult-to-reach area, he can only carry a limited amount of equipment in his field backpack.

Each piece of equipment has a scientific importance value and a specific weight. Dr. Aarav wants to choose the best combination of equipment so that the total scientific value is maximized without exceeding the

backpack's weight capacity.

To solve this efficiently, Dr. Aarav applies the Branch and Bound technique using the 0/1 Knapsack algorithm, which systematically explores possible selections and eliminates combinations that cannot yield better results.

Your task is to help Dr. Aarav determine which equipment weights should be selected to achieve the maximum total value.

Note: The weights must be printed in the same order as they appear in the input.

Input Format

The first line of input contains an integer n, representing the number of available equipment items.

The second line contains n space-separated integers val[i], representing the scientific value of each item.

The third line contains n space-separated integers w[i], representing the weight of each item.

The fourth line contains an integer W, representing the maximum carrying capacity of the backpack.

Output Format

The output displays the "Specified weights: <weights>", where <weights> are the weights of the selected equipment items that maximize the total value using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 4
10 20 30 40
1 2 3 4
8

Output: Specified weights: 1 3 4

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

typedef struct {
    int value, weight, index;
    float ratio;
} Item;

typedef struct {
    int level, profit, weight;
    float bound;
    int taken[MAX];
} Node;

Item items[MAX];
int n, W;
int bestTaken[MAX];
int maxProfit = 0;

// Sort by value/weight ratio (descending)
int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    else return -1;
}

// Calculate upper bound
float bound(Node u) {
    if (u.weight >= W)
        return 0;

    float profit_bound = u.profit;
    int j = u.level + 1;
    int totalWeight = u.weight;
```

```
    while (j < n && totalWeight + items[j].weight <= W) {
        totalWeight += items[j].weight;
        profit_bound += items[j].value;
        j++;
    }

    if (j < n)
        profit_bound += (W - totalWeight) * items[j].ratio;

    return profit_bound;
}

// Branch and Bound
void knapsack() {
    Node u, v;
    Node queue[1000];
    int front = 0, rear = 0;

    u.level = -1;
    u.profit = 0;
    u.weight = 0;
    for (int i = 0; i < n; i++) u.taken[i] = 0;

    u.bound = bound(u);
    queue[rear++] = u;

    while (front < rear) {
        u = queue[front++];

        if (u.level == n - 1)
            continue;

        v.level = u.level + 1;

        // Include item
        v.weight = u.weight + items[v.level].weight;
        v.profit = u.profit + items[v.level].value;

        for (int i = 0; i < n; i++)
            v.taken[i] = u.taken[i];

        v.taken[v.level] = 1;
```

```
        if (v.weight <= W && v.profit > maxProfit) {
            maxProfit = v.profit;

            // Store best solution
            for (int i = 0; i < n; i++)
                bestTaken[i] = v.taken[i];
        }

        v.bound = bound(v);
        if (v.bound > maxProfit)
            queue[rear++] = v;

        // Exclude item
        v.weight = u.weight;
        v.profit = u.profit;

        for (int i = 0; i < n; i++)
            v.taken[i] = u.taken[i];

        v.taken[v.level] = 0;

        v.bound = bound(v);
        if (v.bound > maxProfit)
            queue[rear++] = v;
    }
}

int main() {
    scanf("%d", &n);

    int val[MAX], wt[MAX];

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);

    // Prepare items
```

```
    for (int i = 0; i < n; i++) {
        items[i].value = val[i];
        items[i].weight = wt[i];
        items[i].index = i;
        items[i].ratio = (float)val[i] / wt[i];
    }

    // Sort items by ratio
    qsort(items, n, sizeof(Item), cmp);

    knapsack();

    // Map back to original order
    int result[MAX] = {0};
    for (int i = 0; i < n; i++) {
        if (bestTaken[i])
            result[items[i].index] = 1;
    }

    printf("Specified weights: ");
    for (int i = 0; i < n; i++) {
        if (result[i])
            printf("%d ", wt[i]);
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input consists of an integer n, representing the number of cities.

The next n lines contain n space-separated integers each, representing the distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j.

Output Format

The first line of output displays "The Path is: " followed by the path of the route using ---> in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 4 1 3
4 0 2 1
1 2 0 5
3 1 5 0
Output: The Path is: 1--->3--->2--->4--->1
Minimum cost is 7

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define MAX 10

int n;
```

```
int cost[MAX][MAX];
bool visited[MAX];

int path[MAX], bestPath[MAX];
int minCost = INT_MAX;

// Branch and Bound TSP
void tsp(int currCity, int count, int currCost) {
   // If all cities visited
   if (count == n && cost[currCity][0] > 0) {
      int totalCost = currCost + cost[currCity][0];

      if (totalCost < minCost) {
         minCost = totalCost;

         for (int i = 0; i < n; i++)
            bestPath[i] = path[i];
      }
      return;
   }

   for (int i = 0; i < n; i++) {
      if (!visited[i] && cost[currCity][i] > 0) {
         int newCost = currCost + cost[currCity][i];

         // Pruning
         if (newCost < minCost) {
            visited[i] = true;
            path[count] = i;

            tsp(i, count + 1, newCost);

            visited[i] = false; // backtrack
         }
      }
   }
}

int main() {
   scanf("%d", &n);

   // Input matrix
```

```
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    // Initialize
    for (int i = 0; i < n; i++)
        visited[i] = false;

    visited[0] = true;
    path[0] = 0;

    tsp(0, 1, 0);

    // Print path (1-based indexing)
    printf("The Path is: ");
    for (int i = 0; i < n; i++) {
        printf("%d--->", bestPath[i] + 1);
    }
    printf("1\n");

    printf("Minimum cost is %d", minCost);

    return 0;
}
```

Status : Partially correct **Marks : 8.5/10**

5. Problem Statement

A traveling salesman is tasked with finding the shortest route to visit a set of cities and return to the starting city. The goal is to minimize the total distance traveled by the salesman while visiting each city exactly once.

Input Format

The first line contains an integer n, the number of cities.

The next n lines contain n space-separated integers each, representing the distances between cities.

The ith line represents the distances from city i to all other cities.

The last line contains an integer s, representing the starting city.

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path taken by the salesman, represented as a list of city indices within square brackets.

The second line of output displays "Minimum Cost: " followed by the minimum cost (total distance) of the shortest path.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
0
Output: Shortest Path: [0, 1, 3, 2, 0]
Minimum Cost: 80

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define MAX 10

int n, s;
int cost[MAX][MAX];
bool visited[MAX];

int path[MAX + 1], bestPath[MAX + 1];
int minCost = INT_MAX;

// Branch and Bound TSP
void tsp(int currCity, int count, int currCost) {
```

```
    // If all cities visited
    if (count == n) {
        if (cost[currCity][s] > 0) {
            int totalCost = currCost + cost[currCity][s];

            if (totalCost < minCost) {
                minCost = totalCost;

                for (int i = 0; i < n; i++)
                    bestPath[i] = path[i];

                bestPath[n] = s; // return to start
            }
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[currCity][i] > 0) {
            int newCost = currCost + cost[currCity][i];

            // Pruning
            if (newCost < minCost) {
                visited[i] = true;
                path[count] = i;

                tsp(i, count + 1, newCost);

                visited[i] = false; // backtrack
            }
        }
    }
}

int main() {
    scanf("%d", &n);

    // Input matrix
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
```

```
    scanf("%d", &s);

    // Initialize
    for (int i = 0; i < n; i++)
        visited[i] = false;

    visited[s] = true;
    path[0] = s;

    tsp(s, 1, 0);

    // Print result
    printf("Shortest Path: [");
    for (int i = 0; i <= n; i++) {
        printf("%d", bestPath[i]);
        if (i != n)
            printf(", ");
    }
    printf("]\n");

    printf("Minimum Cost: %d", minCost);

    return 0;
}
```

Status : Correct **Marks : 10/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Riya, the manager of a software development company, needs to assign three developers to three different projects.

Each developer has a different cost (in terms of time and resource usage) for completing each project. Riya wants to assign exactly one developer to each project such that the total development cost is minimized.

To solve this efficiently, she decides to use the Branch and Bound technique for the Assignment Problem.

Your task is to help Riya determine the minimum total cost of assigning developers to projects.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 cost matrix.

Each cell (i, j) represents the cost of assigning the ith developer to the jth project.

Output Format

The output should display the minimum total cost of the assignment in the following exact format: "Minimum total cost: X" , Where X is the minimum total cost obtained after assigning each worker to exactly one task.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10 12 19
10 5 7
15 14 13

Output: Minimum total cost: 28

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define N 3

int cost[N][N];
bool assigned[N];   // Track assigned projects
int minCost = INT_MAX;

// Branch and Bound function
void solve(int worker, int currentCost) {
    // If all workers assigned
    if (worker == N) {
        if (currentCost < minCost)
            minCost = currentCost;
        return;
    }

    // Pruning
```

```
    if (currentCost >= minCost)
        return;

    // Try assigning each project
    for (int j = 0; j < N; j++) {
        if (!assigned[j]) {
            assigned[j] = true;

            solve(worker + 1, currentCost + cost[worker][j]);

            assigned[j] = false; // backtrack
        }
    }
}

int main() {
    // Input cost matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &cost[i][j]);

    // Initialize assigned array
    for (int i = 0; i < N; i++)
        assigned[i] = false;

    solve(0, 0);

    printf("Minimum total cost: %d", minCost);

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Arjun is the manager of a disaster relief center responsible for sending essential supplies to a flood-affected area using a single rescue truck. The truck has a limited carrying capacity, so Arjun must carefully select which supply packages to load.

Each package has a specific weight and an importance score representing how critical it is for the affected people. Arjun wants to maximize the total importance score of the packages loaded onto the truck without exceeding its weight capacity.

To solve this efficiently, Arjun uses the Branch and Bound technique, which systematically explores possible combinations of packages while pruning choices that cannot produce better results.

Your task is to help Arjun determine the maximum total importance score that can be transported.

Input Format

The first line contains an integer N, representing the number of available packages.

The next N lines each contain two integers: weight[i] value[i]

- weight[i] – weight of the package
- value[i] – importance score of the package

The last line contains an integer W, representing the maximum capacity of the rescue truck.

Output Format

The output prints a single integer representing the maximum total importance score that can be carried without exceeding the truck's weight capacity.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5
2 40
3 50
1 100
5 95
3 30

10
Output: 245

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 1000

typedef struct {
    int weight, value;
    float ratio;
} Item;

typedef struct {
    int level, profit, weight;
    float bound;
} Node;

Item items[MAX];
int N, W;

// Comparator for sorting by ratio (descending)
int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    else return -1;
}

// Calculate upper bound
float bound(Node u) {
    if (u.weight >= W)
        return 0;

    float profit_bound = u.profit;
    int j = u.level + 1;
    int totalWeight = u.weight;

    // Take full items
    while (j < N && totalWeight + items[j].weight <= W) {
        totalWeight += items[j].weight;
```

```
        profit_bound += items[j].value;
        j++;
    }

    // Take fraction of next item
    if (j < N)
        profit_bound += (W - totalWeight) * items[j].ratio;

    return profit_bound;
}

// Branch and Bound Knapsack
int knapsack() {
    Node u, v;
    Node queue[MAX];
    int front = 0, rear = 0;

    int maxProfit = 0;

    // Start node
    u.level = -1;
    u.profit = 0;
    u.weight = 0;
    u.bound = bound(u);

    queue[rear++] = u;

    while (front < rear) {
        u = queue[front++];

        if (u.level == N - 1)
            continue;

        // Next level
        v.level = u.level + 1;

        // Include item
        v.weight = u.weight + items[v.level].weight;
        v.profit = u.profit + items[v.level].value;

        if (v.weight <= W && v.profit > maxProfit)
            maxProfit = v.profit;
```

```
        v.bound = bound(v);

        if (v.bound > maxProfit)
            queue[rear++] = v;

        // Exclude item
        v.weight = u.weight;
        v.profit = u.profit;
        v.bound = bound(v);

        if (v.bound > maxProfit)
            queue[rear++] = v;
    }

    return maxProfit;
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        scanf("%d %d", &items[i].weight, &items[i].value);
        items[i].ratio = (float)items[i].value / items[i].weight;
    }

    scanf("%d", &W);

    // Sort items by ratio
    qsort(items, N, sizeof(Item), cmp);

    printf("%d", knapsack());

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city.

Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n, the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from gallery i to gallery j.

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: Minimum Cost is: 80

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>
```

```
#define MAX 10

int n;
int cost[MAX][MAX];
bool visited[MAX];
int minCost = INT_MAX;

// Branch and Bound function
void tsp(int currCity, int count, int currCost) {
    // If all cities visited and return to start
    if (count == n && cost[currCity][0] > 0) {
        int totalCost = currCost + cost[currCity][0];
        if (totalCost < minCost)
            minCost = totalCost;
        return;
    }

    // Try all next cities
    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[currCity][i] > 0) {
            int newCost = currCost + cost[currCity][i];

            // Pruning
            if (newCost < minCost) {
                visited[i] = true;

                tsp(i, count + 1, newCost);

                visited[i] = false; // backtrack
            }
        }
    }
}

int main() {
    scanf("%d", &n);

    // Input cost matrix
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
```

```
    // Initialize visited
    for (int i = 0; i < n; i++)
        visited[i] = false;

    visited[0] = true; // start from city 0

    tsp(0, 1, 0);

    printf("Minimum Cost is: %d", minCost);

    return 0;
}
```

Status : Correct **Marks : 10/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 5

Section 1 : MCQ

1. Which of the following methods is commonly used to solve assignment problems?

Answer

Stepping stone method

Status : Wrong ***Marks : 0/1***

2. The application of assignment problems is to obtain _____

Answer

Minimum cost or maximum profit

Status : Correct ***Marks : 1/1***

3. The assignment problem is said to be balanced if _____.

Answer

the number of rows is equal to the number of columns

Status : Correct ***Marks : 1/1***

4. The minimum number of lines covering all zeros in a reduced cost matrix of order n can be _____.

Answer

at the least n

Status : Wrong ***Marks : 0/1***

5. The assignment problem _____.

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

6. You are given a knapsack that can carry a maximum weight of 60. There are 4 items with weights of {20, 30, 40, and 70} and values {70, 80, 90, and 200}.

What is the maximum value of the items you can carry in the knapsack?

Answer

170

Status : Wrong ***Marks : 0/1***

7. What is the objective of the knapsack problem?

Answer

To Get Minimum Weight In The Knapsack

Status : Wrong ***Marks : 0/1***

8. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

Answer

Using a greedy heuristic to select items.

Status : Wrong ***Marks : 0/1***

9. What is the key difference between Backtracking and Branch and Bound in solving the Knapsack problem?

Answer

Backtracking guarantees an optimal solution, while Branch and Bound does not.

Status : Wrong ***Marks : 0/1***

10. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct ***Marks : 1/1***

11. The travelling salesman problem can be solved using _____.

Answer

Bellman - Ford algorithm

Status : Wrong ***Marks : 0/1***

12. In a traveling salesman problem, the elements of diagonal from left-top to right-bottom are_____.

Answer

all infinity

Status : Correct ***Marks : 1/1***

13. Which type of graph is used to model the TSP problem?

Answer

Directed Unweighted Graph

Status : Wrong ***Marks : 0/1***

14. What is the main goal of using Branch and Bound in TSP?

Answer

To find the longest tour

Status : Wrong ***Marks : 0/1***

15. Which value helps in deciding whether a node should be pruned in Branch and Bound?

Answer

Tour length

Status : Wrong ***Marks : 0/1***

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an adjacency matrix adj[][] of an undirected graph consisting of N vertices, find a Hamiltonian Path in the graph if one exists. If found, print the path as a sequence of vertex indices separated by ->. Otherwise, print No Hamiltonian Path found.

A Hamiltonian path visits each and every vertex of the graph exactly once. The adjacency matrix is symmetric with zeros on the diagonal (no self-loops).

Write a program using Backtracking to find the Hamiltonian Path.

Examples:

Input: adj[][] = {{0, 1, 1, 1, 0}, {1, 0, 1, 0, 1}, {1, 1, 0, 1, 1}, {1, 0, 1, 0, 0}}

Output: Yes

Explanation:

There exists a Hamiltonian Path for the given graph, as shown in the image below:

Input: adj[][] = {{0, 1, 0, 0}, {1, 0, 1, 1}, {0, 1, 0, 0}, {0, 1, 0, 0}}

Output: No

Input Format

The first line contains an integer N, representing the number of vertices in the graph.

The next N lines each contain N space-separated integers (0 or 1), representing the adjacency matrix of the graph.

Output Format

If a Hamiltonian path exists, print the vertices of the path separated by -> on a single line.

If no Hamiltonian path exists, print: No Hamiltonian Path found

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 1 1 1 0
1 0 1 0 1
1 1 0 1 1
1 0 1 0 0
0 1 1 0 0
Output: 0 -> 1 -> 4 -> 2 -> 3

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>

#define MAX 10

int N;
int adj[MAX][MAX];
bool visited[MAX];
int path[MAX];

// Function to check if vertex v can be added at position pos
bool isSafe(int v, int pos) {
    // Check if there is an edge from previous vertex
    if (adj[path[pos - 1]][v] == 0)
        return false;

    // Check if already visited
    if (visited[v])
        return false;

    return true;
}

// Backtracking function to find Hamiltonian Path
bool hamiltonianPath(int pos) {
    // If all vertices are included
    if (pos == N)
        return true;

    // Try different vertices
    for (int v = 1; v < N; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;
            visited[v] = true;

            if (hamiltonianPath(pos + 1))
                return true;

            // Backtracking
            visited[v] = false;
        }
    }
    return false;
```

```
}
int main() {
    scanf("%d", &N);

    // Read adjacency matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &adj[i][j]);

    // Initialize
    for (int i = 0; i < N; i++)
        visited[i] = false;

    // Start from vertex 0
    path[0] = 0;
    visited[0] = true;

    if (hamiltonianPath(1)) {
        for (int i = 0; i < N; i++) {
            printf("%d", path[i]);
            if (i != N - 1)
                printf(" -> ");
        }
    } else {
        printf("No Hamiltonian Path found");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

You are given an undirected graph with N vertices. Implement a function longest_hamiltonian_cycle to find the longest Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same.

Input Format

The first line of input consists of an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at graph[i][j] is 1 if there is an edge between vertex i and vertex j, and 0 otherwise.

Output Format

If a Hamiltonian cycle exists, print "Longest Hamiltonian Cycle:" followed by the vertices of the longest cycle found in order within square bracket separated by a comma.

If no Hamiltonian cycle exists, print "No Hamiltonian Cycle found".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 1 1 0
1 0 1 1
1 1 0 1
0 1 1 0
Output: Longest Hamiltonian Cycle: [0, 1, 3, 2]

Answer

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 10

int N;
int graph[MAX][MAX];
int path[MAX];
bool visited[MAX];

// Check if vertex v can be added at position pos
bool isSafe(int v, int pos) {
```

```
    if (graph[path[pos - 1]][v] == 0)
        return false;

    if (visited[v])
        return false;

    return true;
}

// Backtracking function
bool hamiltonianCycle(int pos) {
    // If all vertices are included
    if (pos == N) {
        // Check if last vertex connects to first
        if (graph[path[pos - 1]][path[0]] == 1)
            return true;
        else
            return false;
    }

    for (int v = 1; v < N; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;
            visited[v] = true;

            if (hamiltonianCycle(pos + 1))
                return true;

            // Backtrack
            visited[v] = false;
        }
    }
    return false;
}

int main() {
    scanf("%d", &N);

    // Input adjacency matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &graph[i][j]);
```

```
    // Initialize
    for (int i = 0; i < N; i++)
        visited[i] = false;

    path[0] = 0;
    visited[0] = true;

    if (hamiltonianCycle(1)) {
        printf("Longest Hamiltonian Cycle: [");
        for (int i = 0; i < N; i++) {
            printf("%d", path[i]);
            if (i != N - 1)
                printf(", ");
        }
        printf("]");
    } else {
        printf("No Hamiltonian Cycle found");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Nira is studying the N-Queens problem and needs to write a program to solve it. The program should find a valid arrangement of N queens on an N×N chessboard, ensuring that no two queens can attack each other.

Two queens attack each other if they are in the same row, the same column, or the same diagonal. Once a solution is found, display the board with queens placed correctly. A solution is guaranteed to exist for all valid values of N.

Write a program using Backtracking to solve the N-Queens problem.

Example

Input:

4 // Value of N

Output:

0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

Explanation: The output represents a valid arrangement of queens on the 4x4 chessboard, where 1 indicates the position of a queen and 0 indicates an empty cell.

In the first row, a queen is placed in the third column.In the second row, a queen is placed in the first column.In the third row, a queen is placed in the fourth column.In the fourth row, a queen is placed in the second column.This arrangement of queens on the board ensures that no two queens threaten each other.

Input Format

The input consists of a single integer N, representing the size of the chessboard (N×N).

Output Format

Print the N×N board as N rows of N space-separated integers. Each integer is followed by a space (including the last integer on each row). Print 1 where a queen is placed and 0 for an empty cell. Print any one valid solution.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

Output: 0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

Answer

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 20

int N;
int board[MAX][MAX];

// Check if it's safe to place queen at board[row][col]
bool isSafe(int row, int col) {
    int i, j;

    // Check column
    for (i = 0; i < row; i++)
        if (board[i][col])
            return false;

    // Check left diagonal
    for (i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return false;

    // Check right diagonal
    for (i = row - 1, j = col + 1; i >= 0 && j < N; i--, j++)
        if (board[i][j])
            return false;

    return true;
}

// Backtracking function
bool solveNQueens(int row) {
    if (row == N)
        return true;

    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row][col] = 1;

            if (solveNQueens(row + 1))
                return true;
```

```
            // Backtrack
            board[row][col] = 0;
        }
    }
    return false;
}

int main() {
    scanf("%d", &N);

    // Initialize board
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;

    solveNQueens(0);

    // Print solution
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

4. Problem Statement

In chess, a knight moves in an "L" shape: two squares in one direction and one square perpendicular to it. From position (0, 0) on an infinite chessboard, a knight can move to one of the following eight positions in a single step:

(1, 2), (−1, 2), (1, −2), (−1, −2), (2, 1), (−2, 1), (2, −1), (−2, −1).

Given a target position (x, y) where both x and y are positive integers, find

the minimum number of steps needed for the knight to travel from (0, 0) to (x, y) using BFS (Breadth-First Search).

Beginning from location (0,0), what is the minimum number of steps needed for a knight to get to some other arbitrary location (x,y)?

Input Format

The first line contains two space-separated integers x and y, representing the target position of the knight on the chessboard.

Output Format

Print a single integer representing the minimum number of steps needed for the knight to move from (0, 0) to (x, y).

Refer to the sample output for formatting specifications

Sample Test Case

Input: 1 2

Output: 1

Answer

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 20

// Structure for queue node
typedef struct {
    int x, y, dist;
} Node;

// All 8 possible knight moves
int dx[] = {1, -1, 1, -1, 2, -2, 2, -2};
int dy[] = {2, 2, -2, -2, 1, 1, -1, -1};

// Function to find minimum steps
```

```
int minSteps(int targetX, int targetY) {
    bool visited[MAX][MAX] = {false};

    Node queue[MAX * MAX];
    int front = 0, rear = 0;

    // Start from (0,0)
    queue[rear++] = (Node){0, 0, 0};
    visited[0][0] = true;

    while (front < rear) {
        Node curr = queue[front++];

        // If target reached
        if (curr.x == targetX && curr.y == targetY)
            return curr.dist;

        // Try all moves
        for (int i = 0; i < 8; i++) {
            int nx = curr.x + dx[i];
            int ny = curr.y + dy[i];

            // Check bounds and visited
            if (nx >= 0 && ny >= 0 && nx < MAX && ny < MAX && !visited[nx][ny]) {
                visited[nx][ny] = true;
                queue[rear++] = (Node){nx, ny, curr.dist + 1};
            }
        }
    }
    return -1; // Should not happen
}

int main() {
    int x, y;
    scanf("%d %d", &x, &y);

    printf("%d", minSteps(x, y));

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

A customer is shopping in a supermarket and has selected N items, each with a fixed price.

The customer has a discount coupon that can be applied only if the total price of some selected items is exactly equal to a target amount S.

Write a program using Backtracking to find all distinct subsets of item prices whose sum is exactly equal to S.

Each item price can be used only once. If the input contains duplicate prices, the output must not contain repeated subsets.

Formula: A subset is valid if: p1 + p2 + ... + pk = S

Print each valid subset with its elements in ascending order. Print subsets in the order they are discovered by the backtracking algorithm (which processes the sorted array from left to right).

Input Format

The first line contains an integer N representing the number of items.

The second line contains N space-separated integers representing item prices. The prices may include duplicate values.

The third line contains an integer S representing the target bill amount.

Output Format

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last element).

Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 5 10 15 20
25

Output: 5 5 15
5 20
10 15

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

int N, S;
int arr[MAX];
int subset[MAX];
int found = 0;

// Comparator for sorting
int cmp(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

// Backtracking function
void findSubsets(int index, int sum, int size) {
    // If sum equals target, print subset
    if (sum == S) {
        found = 1;
        for (int i = 0; i < size; i++)
            printf("%d ", subset[i]);
        printf("\n");
        return;
    }

    // Explore further elements
    for (int i = index; i < N; i++) {
        // Skip duplicates
        if (i > index && arr[i] == arr[i - 1])
            continue;
```

```
        // Pruning
        if (sum + arr[i] > S)
            break;

        subset[size] = arr[i];
        findSubsets(i + 1, sum + arr[i], size + 1);
    }
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &S);

    // Sort array
    qsort(arr, N, sizeof(int), cmp);

    findSubsets(0, 0, 0);

    if (!found)
        printf("No subset found");

    return 0;
}
```

Status : Correct **Marks : 10/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 39.5

Section 1 : Coding

1. Problem Statement

Diya is given a weighted, undirected graph with N vertices. She wants to find the minimum weight Hamiltonian cycle in the graph, if one exists. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same. The weight of a Hamiltonian cycle is the sum of the weights of all edges in the cycle:

W = w(v , v) + w(v , v) + ... + w(v , v)

Write a program using Backtracking to find the minimum weight Hamiltonian cycle.

Example: For a graph with 4 vertices and the adjacency matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

The algorithm explores all Hamiltonian cycles starting from vertex 0. Among all valid cycles, 0 1 3 2 0 has the minimum weight: 10 + 25 + 30 + 15 = 80.

Input Format

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers representing the adjacency matrix of the weighted graph. The value at graph[i][j] is the weight of the edge between vertices i and j. A value of 0 means no edge exists. The matrix is symmetric (graph[i][j] == graph[j][i]) and the diagonal is always 0 (graph[i][i] == 0).

Output Format

If a minimum weight Hamiltonian cycle exists, print Minimum Weight Hamiltonian Cycle: on the first line, then print the vertices of the cycle in square brackets separated by a comma and a space, including the starting vertex at both the beginning and the end of the cycle. On the next line, print Total Weight: followed by a space and the integer total weight.

If no Hamiltonian cycle exists, print No Hamiltonian Cycle found with minimum weight.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: Minimum Weight Hamiltonian Cycle:
[0, 1, 3, 2, 0]
Total Weight: 80

Answer

```
#include <stdio.h>
#include <limits.h>

#define MAX 10

int N;
int graph[MAX][MAX];

int path[MAX], bestPath[MAX];
int visited[MAX];

int minCost = INT_MAX;

// Backtracking function
void tsp(int pos, int count, int cost) {

   // If all vertices visited
   if (count == N && graph[pos][0] != 0) {
      int totalCost = cost + graph[pos][0];

      if (totalCost < minCost) {
         minCost = totalCost;

         for (int i = 0; i < N; i++)
            bestPath[i] = path[i];
      }
      return;
   }

   for (int i = 0; i < N; i++) {
      if (!visited[i] && graph[pos][i] != 0) {

         visited[i] = 1;
         path[count] = i;

         tsp(i, count + 1, cost + graph[pos][i]);
```

```
            visited[i] = 0; // backtrack
        }
    }
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &graph[i][j]);

    // Initialize
    for (int i = 0; i < N; i++)
        visited[i] = 0;

    visited[0] = 1;
    path[0] = 0;

    tsp(0, 1, 0);

    if (minCost == INT_MAX) {
        printf("No Hamiltonian Cycle found");
    } else {
        printf("Minimum Weight Hamiltonian Cycle:\n[");

        for (int i = 0; i < N; i++) {
            printf("%d", bestPath[i]);
            if (i < N - 1) printf(", ");
        }
        printf(", 0]\n");

        printf("Total Weight: %d", minCost);
    }

    return 0;
}
```

Status : Partially correct **Marks : 8.5/10**

2. Problem Statement

Alice is tasked with determining whether a given graph contains a Hamiltonian Cycle and, if it does, listing all such cycles. A Hamiltonian Cycle in a graph is a cycle that visits each vertex exactly once and returns to the starting vertex.

Write a program using Backtracking that takes a graph represented by an adjacency matrix and finds all Hamiltonian Cycles starting from the first vertex (vertex 0), if any exist.

Input Format

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain a single string — the name of each vertex (no surrounding spaces).

The following N lines each contain N space-separated integers (0 or 1) representing the adjacency matrix. A value of 1 at position [i][j] means an edge exists between vertex i and vertex j; 0 means no edge.

Output Format

For each Hamiltonian Cycle found, print a line in the format: ['v0', 'v1', ..., 'vn', 'v0'] where vertex names are enclosed in single quotes, separated by a comma and a space, with the starting vertex repeated at the end. Print cycles in the order they are discovered by the backtracking algorithm.

If no Hamiltonian Cycle exists, print: No Hamiltonian Cycle

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8
a
b

c
d
e
f
g
h
0 1 0 0 0 0 0 1
1 0 1 0 0 0 0 0
0 1 0 1 0 0 0 1
0 0 1 0 1 0 1 0
0 0 0 1 0 1 0 0
0 0 0 0 1 0 1 0
0 0 0 1 0 1 0 1
1 0 1 0 0 0 1 0
Output: ['a', 'b ', 'c ', 'd ', 'e ', 'f ', 'g ', 'h', 'a']
['a', 'h', 'g ', 'f ', 'e ', 'd ', 'c ', 'b ', 'a']

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 10

int N;
int graph[MAX][MAX];
char names[MAX][20];

int path[MAX];
int visited[MAX];
int found = 0;

// Backtracking function
void findCycles(int pos, int count) {

    // If all vertices visited
    if (count == N) {
        // Check edge back to start
        if (graph[pos][0] == 1) {
            found = 1;

            printf("[");
```

```
            for (int i = 0; i < N; i++) {
                printf("'%s', ", names[path[i]]);
            }
            printf("'%s']\n", names[0]); // back to start
        }
        return;
    }

    for (int i = 1; i < N; i++) {
        if (!visited[i] && graph[pos][i] == 1) {
            visited[i] = 1;
            path[count] = i;

            findCycles(i, count + 1);

            visited[i] = 0; // backtrack
        }
    }
}

int main() {
    scanf("%d", &N);

    // Read vertex names (including spaces if any)
    for (int i = 0; i < N; i++) {
        scanf(" %[^\n]", names[i]);
    }

    // Read adjacency matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &graph[i][j]);

    // Initialize
    for (int i = 0; i < N; i++)
        visited[i] = 0;

    path[0] = 0;
    visited[0] = 1;

    findCycles(0, 1);
```

```
    if (!found) {
        printf("No Hamiltonian Cycle");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

A company has N employees, and each employee can contribute a fixed number of training hours.

The manager wants to form groups of employees such that the total training hours of a selected group is exactly equal to the required training goal S.

Write a program using Backtracking to find all distinct subsets of employee training hours whose sum is exactly equal to S.

Each employee's training hours can be used only once.

If the input contains duplicate hour values, the output must not contain repeated subsets.

Condition

A subset is valid if:

h1 + h2 + ... + hk = S

Input Format

The first line contains an integer N representing the number of employees.

The second line contains N space-separated integers representing the training hours of each employee. The input may contain duplicate values.

The third line contains an integer S representing the required training goal.

Output Format

Print each distinct valid subset on a separate line. Within each subset, print

elements in ascending order, each followed by a space (including the last element). Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 2 3 5 6 8
10
Output: 2 2 6
2 3 5
2 8

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

int found = 0;

// Comparison function for sorting
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

// Backtracking function
void findSubsets(int arr[], int n, int start, int sum, int target, int subset[], int size) {

    if (sum == target) {
        found = 1;
        for (int i = 0; i < size; i++) {
            printf("%d ", subset[i]);
        }
        printf("\n");
        return;
    }
```

```
    if (sum > target) return;

    for (int i = start; i < n; i++) {

        // Skip duplicates
        if (i > start && arr[i] == arr[i - 1]) continue;

        subset[size] = arr[i];

        findSubsets(arr, n, i + 1, sum + arr[i], target, subset, size + 1);
    }
}

int main() {
    int n, S;
    int arr[MAX], subset[MAX];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &S);

    // Sort array
    qsort(arr, n, sizeof(int), compare);

    findSubsets(arr, n, 0, 0, S, subset, 0);

    if (!found) {
        printf("No subset found");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Nikila is exploring the N Queens Problem. This classic problem involves placing N queens on an N×N chessboard such that no two queens threaten each other. In chess, queens can move horizontally, vertically, and diagonally, so the challenge is to arrange the queens to ensure they do not share the same row, column, or diagonal.

Write a program using Backtracking to find and print all valid solutions to the N Queens Problem.

Input Format

The input consists of a single integer N representing both the size of the chessboard (N×N) and the number of queens to be placed.

Output Format

Print all valid solutions in the order they are found by the backtracking algorithm. Each solution is represented as an N×N matrix where 1 indicates the position of a queen and 0 represents an empty cell. Each value is followed by a space. Each row is printed on a new line. Print a blank line after each solution including the last one. If no valid solution exists, print nothing.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
Output: 0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Answer

```
#include <stdio.h>
```

```
#define MAX 10

int N;
int board[MAX][MAX];

// Check if safe
int isSafe(int row, int col) {

    // Check column
    for (int i = 0; i < row; i++) {
        if (board[i][col] == 1)
            return 0;
    }

    // Left diagonal
    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] == 1)
            return 0;
    }

    // Right diagonal
    for (int i = row - 1, j = col + 1; i >= 0 && j < N; i--, j++) {
        if (board[i][j] == 1)
            return 0;
    }

    return 1;
}

// Backtracking
void solve(int row) {

    if (row == N) {
        // Print solution
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                printf("%d ", board[i][j]);
            }
            printf("\n");
        }
        printf("\n");
```

```
        return;
    }

    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row][col] = 1;

            solve(row + 1);

            board[row][col] = 0; // backtrack
        }
    }
}

int main() {
    scanf("%d", &N);

    // Initialize board
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;

    solve(0);

    return 0;
}
```

Status : Partially correct **Marks : 1/10**

5. Problem Statement

Sarah wants to verify whether a given arrangement of queens on a 4×4 chessboard constitutes a valid solution to the 4 Queens Problem. The objective is to position four queens on the board in a way that prevents any queen from threatening another, considering horizontal, vertical, and diagonal movements.

To determine the validity of the solution, Sarah needs to ensure that there are exactly four queens on the board, each row and column contains only one queen, and no two queens share the same diagonal.

Your task is to assist Sarah by implementing a program that checks these criteria for the given chessboard arrangement.

Input Format

The input consists of 4 lines, each containing 4 space-separated integers representing one row of the chessboard. Each integer is either 1 (queen present) or 0 (empty cell).

Output Format

Print a single line indicating whether the placement is valid, checking conditions in the following priority order:

1. If the total number of queens is not exactly 4: False - Invalid number of queens: X where X is the count of queens on the board.
2. If a row does not contain exactly one queen: False - Row Y has Z queens where Y is the 0-based row index and Z is the number of queens in that row.
3. If a column does not contain exactly one queen: False - Column Y has Z queens where Y is the 0-based column index and Z is the number of queens in that column.
4. If a main diagonal (top-left to bottom-right) contains more than one queen: False - Main diagonal (d=X) has Y queens where X is the diagonal identifier computed as column - row (range −3 to 3) and Y is the number of queens on that diagonal.
5. If an anti-diagonal (top-right to bottom-left) contains more than one queen: False - Anti-diagonal (d=X) has Y queens where X is the diagonal identifier computed as column + row (range 0 to 6) and Y is the number of queens on that diagonal.
6. If all conditions are satisfied: True - Valid 4 Queens solution: All conditions satisfied

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
Output: True - Valid 4 Queens solution: All conditions satisfied

Answer

```
#include <stdio.h>

int main() {
    int board[4][4];
    int total = 0;

    // Input
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            scanf("%d", &board[i][j]);
            if (board[i][j] == 1)
                total++;
        }
    }

    // 1. Check total queens
    if (total != 4) {
        printf("False - Invalid number of queens: %d", total);
        return 0;
    }

    // 2. Check rows
    for (int i = 0; i < 4; i++) {
        int rowCount = 0;
        for (int j = 0; j < 4; j++) {
            if (board[i][j] == 1)
                rowCount++;
        }
        if (rowCount != 1) {
            printf("False - Row %d has %d queens", i, rowCount);
            return 0;
        }
    }

    // 3. Check columns
    for (int j = 0; j < 4; j++) {
        int colCount = 0;
        for (int i = 0; i < 4; i++) {
            if (board[i][j] == 1)
                colCount++;
        }
```

```
        if (colCount != 1) {
            printf("False - Column %d has %d queens", j, colCount);
            return 0;
        }
    }

    // 4. Main diagonals (col - row)
    for (int d = -3; d <= 3; d++) {
        int count = 0;
        for (int i = 0; i < 4; i++) {
            int j = i + d;
            if (j >= 0 && j < 4 && board[i][j] == 1)
                count++;
        }
        if (count > 1) {
            printf("False - Main diagonal (d=%d) has %d queens", d, count);
            return 0;
        }
    }

    // 5. Anti-diagonals (col + row)
    for (int d = 0; d <= 6; d++) {
        int count = 0;
        for (int i = 0; i < 4; i++) {
            int j = d - i;
            if (j >= 0 && j < 4 && board[i][j] == 1)
                count++;
        }
        if (count > 1) {
            printf("False - Anti-diagonal (d=%d) has %d queens", d, count);
            return 0;
        }
    }

    // 6. Valid
    printf("True - Valid 4 Queens solution: All conditions satisfied");

    return 0;
}
```

Status : Correct **Marks : 10/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Bob has a task to find a Hamiltonian cycle in a given undirected simple graph (no self-loops) represented as an adjacency matrix, where a value of 1 indicates an edge exists between two vertices and a value of 0 indicates no edge. A Hamiltonian cycle is a cycle that visits every vertex exactly once and returns to the starting vertex. Bob needs to write a program that will determine if such a cycle exists and, if so, print the cycle starting and ending at vertex 0. If no cycle is found, the program should output "No solution".

Your task is to help Bob implement this.

Input Format

The first line contains an integer V, representing the number of vertices.

The following V lines each contain V space-separated integers (0 or 1), representing the adjacency matrix. The matrix is symmetric (undirected graph) and has zeros on the diagonal (no self-loops)

Output Format

If a Hamiltonian cycle is found, the first line prints "Solution found" and the second line prints the cycle as a series of space-separated integers, representing the vertices starting and ending at vertex 0.

If no Hamiltonian cycle is found, print "No solution".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8
0 1 0 1 1 0 0 0
1 0 1 0 0 1 0 0
0 1 0 1 0 0 1 0
1 0 1 0 0 0 0 1
1 0 0 0 0 1 0 1
0 1 0 0 1 0 1 0
0 0 1 0 0 1 0 1
0 0 0 1 1 0 1 0
Output: Solution found
0 1 2 3 7 6 5 4 0

Answer

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_V 8

int V;
int graph[MAX_V][MAX_V];
int path[MAX_V + 1];

// Function to check if the vertex v can be added at index 'pos' in the Hamiltonian
Cycle
```

```
bool isSafe(int v, int pos) {
    // Check if this vertex is adjacent to the previously added vertex
    if (graph[path[pos - 1]][v] == 0)
        return false;

    // Check if the vertex has already been included in the path
    for (int i = 0; i < pos; i++)
        if (path[i] == v)
            return false;

    return true;
}

// Recursive function to solve the Hamiltonian Cycle problem
bool hamCycleUtil(int pos) {
    // Base case: If all vertices are included in the path
    if (pos == V) {
        // Check if there is an edge from the last vertex back to the starting vertex
        if (graph[path[pos - 1]][path[0]] == 1)
            return true;
        else
            return false;
    }

    // Try different vertices as the next candidate in the Hamiltonian Cycle
    for (int v = 1; v < V; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;

            if (hamCycleUtil(pos + 1))
                return true;

            // Backtrack if adding vertex v doesn't lead to a solution
            path[pos] = -1;
        }
    }

    return false;
}

void solveHamCycle() {
    // Initialize path array
```

```
    for (int i = 0; i < V + 1; i++)
        path[i] = -1;

    // Start at vertex 0
    path[0] = 0;

    if (hamCycleUtil(1) == false) {
        printf("No solution\n");
        return;
    }

    // Set the last element to 0 to complete the cycle
    path[V] = 0;

    printf("Solution found\n");
    for (int i = 0; i <= V; i++) {
        printf("%d%s", path[i], (i == V) ? "" : " ");
    }
    printf("\n");
}

int main() {
    if (scanf("%d", &V) != 1) return 0;

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    solveHamCycle();

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Given an undirected complete graph with N vertices (N ≥ 3), find the number of distinct Hamiltonian cycles in the graph using backtracking. A

Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex.

Two Hamiltonian cycles are considered identical if one can be obtained from the other by rotation (changing the starting vertex) or reversal (traversing in the opposite direction).

Example:

For N=4, the vertices are 0, 1, 2, 3. The 3 distinct Hamiltonian cycles are:

0 1 2 3 00 1 3 2 00 2 1 3 0

Output: 3

Input Format

The input consists of a single integer N, representing the number of vertices in the complete graph.

Output Format

Print a single integer on its own line representing the number of distinct Hamiltonian cycles in the complete graph with N vertices.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 60

Answer

```
// You are using GCC
#include <stdio.h>

// Function to calculate factorial
long long factorial(int n) {
    long long fact = 1;
    for(int i = 1; i <= n; i++) {
        fact = fact * i;
    }
```

```
    return fact;
}

int main() {
    int N;
    scanf("%d", &N);

    long long result = factorial(N - 1) / 2;

    printf("%lld\n", result);

    return 0;
}
```

Status : Correct *Marks : 10/10*

3. Problem Statement

A college is organizing an event and has collected donations from N sponsors. Each sponsor contributes a fixed amount. The event coordinator wants to know all possible groups of sponsors whose donation sum matches exactly the required event budget S.

Write a program using Backtracking to find all subsets of sponsor contributions whose sum is exactly equal to S.

A subset may contain any number of sponsors, but each sponsor amount can be used only once.

Formula:

A subset is valid if: a1 + a2 + ... + ak = S

Input Format

The first line contains an integer N, representing the number of sponsors.

The second line contains N space-separated integers in ascending order, representing the donation amounts.

The third line contains an integer S, representing the required budget.

Output Format

Print all subsets whose sum is exactly S, one subset per line. Within each subset, print the elements in the order they appear in the input, each followed by a space. Subsets are printed in the order they are discovered during backtracking (subsets starting with smaller elements appear first).

If no subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
10 20 30 40
50

Output: 10 40
20 30

Answer

```
#include <stdio.h>

int found = 0;

// Function for backtracking
void subsetSum(int arr[], int n, int index, int sum, int target, int subset[], int size) {

    // If sum matches target, print subset
    if (sum == target) {
        found = 1;
        for (int i = 0; i < size; i++) {
            printf("%d ", subset[i]);
        }
        printf("\n");
        return;
    }

    // If sum exceeds or index out of range
    if (sum > target || index == n) {
```

```
        return;
    }

    // Include current element
    subset[size] = arr[index];
    subsetSum(arr, n, index + 1, sum + arr[index], target, subset, size + 1);

    // Exclude current element
    subsetSum(arr, n, index + 1, sum, target, subset, size);
}

int main() {
    int n, S;

    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &S);

    int subset[n];

    subsetSum(arr, n, 0, 0, S, subset, 0);

    if (!found) {
        printf("No subset found");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each

other, with the condition that the queen at the specified position cannot be moved.

A queen can attack any cell in its row, column, or diagonal. Row and column indices are 0-based.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j, representing the column index of the fixed queen.

Output Format

The first line of output consists of a single floating-point number representing the maximum total value, formatted to two decimal places.

The next n lines each consist of a single floating-point number representing the fraction x of each item selected, formatted to two decimal places. x represents the fraction of the i-th item taken (1.00 means the item is fully taken, a value between 0.00 and 1.00 means a fraction is taken). The fractions should be printed in the same order as the input items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0
1
Output: . Q . .
. . . Q
Q . . .
. . Q .

Answer

```
#include <stdio.h>

#define MAX 10

int N;
char board[MAX][MAX];
int fixedRow, fixedCol;
int found = 0;

// Check if safe to place queen
int isSafe(int row, int col) {
    // Check column
    for (int i = 0; i < row; i++) {
        if (board[i][col] == 'Q')
            return 0;
    }

    // Check left diagonal
    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] == 'Q')
            return 0;
    }

    // Check right diagonal
    for (int i = row - 1, j = col + 1; i >= 0 && j < N; i--, j++) {
        if (board[i][j] == 'Q')
            return 0;
    }

    return 1;
}

// Backtracking function
void solve(int row) {
    if (row == N) {
        found = 1;
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                printf("%c ", board[i][j]);
```

```
            }
            printf("\n");
        }
        printf("\n");
        return;
    }

    // Skip fixed queen row
    if (row == fixedRow) {
        solve(row + 1);
        return;
    }

    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row][col] = 'Q';
            solve(row + 1);
            board[row][col] = '.';
        }
    }
}

int main() {
    scanf("%d", &N);
    scanf("%d", &fixedRow);
    scanf("%d", &fixedCol);

    // Initialize board
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = '.';

    // Place fixed queen
    board[fixedRow][fixedCol] = 'Q';

    solve(0);

    if (!found) {
        printf("No solution");
    }

    return 0;
```

}

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 8

Section 1 : MCQ

1. The problem of finding a path in a graph that visits every vertex exactly once is called?

Answer

Hamiltonian path problem

Status : Correct ***Marks : 1/1***

2. In the Hamiltonian Cycle problem, if a graph contains a Hamiltonian cycle, it is called a

Answer

Hamiltonian graph

Status : Correct ***Marks : 1/1***

3. What is the primary difficulty in finding a Hamiltonian Path in a general graph?

Answer

It is an NP-complete problem.

Status : Correct ***Marks : 1/1***

4. Which of the following statements is true regarding the backtracking algorithm for the Hamiltonian Cycle problem?

Answer

It explores different paths in the graph to find a Hamiltonian cycle.

Status : Correct ***Marks : 1/1***

5. Which algorithmic paradigm is commonly used to solve the Hamiltonian Cycle problem?

Answer

Backtracking

Status : Correct ***Marks : 1/1***

6. The following paradigm can be used to find the solution to the problem in minimum time: Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K:

Answer

Dynamic Programming

Status : Correct ***Marks : 1/1***

7. What is a subset sum problem?

Answer

Finding a subset of a set that has the sum of elements equal to a given number

Status : Wrong ***Marks : 0/1***

8. Which of the following is not true about the subset sum problem?

Answer

the recursive solution is slower than the dynamic programming solution

Status : Wrong ***Marks : 0/1***

9. What is the worst case time complexity of dynamic programming solution of the subset sum problem(sum=given subset sum)?

Answer

O(sum)

Status : Wrong ***Marks : 0/1***

10. Placing N-queens so that no two queens attack each other is called _____.

Answer

Hamiltonian circuit problem

Status : Wrong ***Marks : 0/1***

11. Where is the n-queens problem implemented?

Answer

Carom

Status : Wrong ***Marks : 0/1***

12. Which of the following methods can be used to solve N-Queen's problem?

Answer

Backtracking

Status : Correct ***Marks : 1/1***

13. Which one of the following is an application of the backtracking algorithm?

Answer

Finding the efficient quantity to shop

Status : Wrong ***Marks : 0/1***

14. For N Queens problem, the time complexity is

Answer

O(NM)

Status : Wrong ***Marks : 0/1***

15. How many unique solutions are there to the 4 Queens problem, where solutions that are reflections or rotations of each other are considered the same?

Answer

2

Status : Correct ***Marks : 1/1***

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 21

Section 1 : Coding

1. Problem Statement

You are tasked with finding the shortest communication paths from a source router to all other routers in a network using Dijkstra's algorithm.

The network is represented as a graph, where routers are nodes and communication links between them are edges with weights representing the time or cost of communication. A value of 0 in the adjacency matrix indicates no direct link between two routers.

Example

Explanation:

The minimum distance from 0 to 1 = 4. Path: 0 1

The minimum distance from 0 to 2 = 12. Path: 0 1 2

The minimum distance from 0 to 3 = 19. Path: 0 1 2 3

The minimum distance from 0 to 4 = 21. Path: 0 7 6 5 4

The minimum distance from 0 to 5 = 11. Path: 0 7 6 5

The minimum distance from 0 to 6 = 9. Path: 0 7 6

The minimum distance from 0 to 7 = 8. Path: 0 7

The minimum distance from 0 to 8 = 14. Path: 0 1 2 8"

Input Format

The first line contains an integer V, representing the number of vertices in the graph.

The next V lines each contain V space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct edge between the vertices.

The last line contains an integer src, representing the source vertex.

Output Format

The first line of output prints: Vertex followed by a tab character followed by Distance from Source

The next V lines each print the vertex number followed by three tab characters, a space, and the shortest distance from the source. Vertices are printed in ascending order from 0 to V-1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 1 3 1
1 0 4 2
3 4 0 5
1 2 5 0

1

Output: Vertex Distance from Source

0 1

1 0

2 4

3 2

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define MAXN 100
#define INF INT_MAX

int N;
int graph[MAXN][MAXN];

int minDistance(int dist[], bool visited[]) {
    int min = INF, minIndex = -1;
    for (int v = 0; v < N; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void dijkstra(int src) {
    int dist[MAXN];
    bool visited[MAXN];

    for (int i = 0; i < N; i++) {
        dist[i] = INF;
        visited[i] = false;
    }
    dist[src] = 0;

    for (int count = 0; count < N - 1; count++) {
        int u = minDistance(dist, visited);
        if (u == -1) break; // no more reachable vertices
```

```
        visited[u] = true;

        for (int v = 0; v < N; v++) {
            if (!visited[v] && graph[u][v] != 0 && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    // Print results
    printf("Vertex Distance from Source\n");
    for (int i = 0; i < N; i++) {
        printf("%d\t\t%d\n", i, dist[i]);
    }
}

int main() {
    scanf("%d", &N);

    // Read adjacency matrix
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int src;
    scanf("%d", &src);

    dijkstra(src);

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Given a network of cities represented as an unweighted graph, find the shortest path between a source city and a destination city in terms of the

minimum number of roads to be traversed. Implement a program that utilizes the Dijkstra algorithm to solve this problem.

The program should take the city network, source city, and destination city as inputs, and provide the shortest path length and the sequence of cities to be visited along the path as outputs.

If no path exists between the source and destination, print 'No path found'.
Note: Since the graph is unweighted, all edges have an implicit weight of 1.

Input Format

The first line contains an integer n, representing the number of cities in the network.

The second line contains an integer m, representing the number of roads in the city network.

The following m lines contain pairs of integers u and v, representing a road between cities u and v.

The next line contains an integer s, representing the source city.

The last line contains an integer d, representing the destination city.

Output Format

The first line prints Shortest path length: followed by the number of edges in the shortest path.

The second line prints Path: followed by the city indices along the shortest path, each followed by a space.

If no path exists, print No path found.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8
10

0 1
0 3
1 2
3 4
3 7
4 5
4 6
4 7
5 6
6 7
0
7

Output: Shortest path length: 2
Path: 0 3 7

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define MAXN 100
#define INF INT_MAX

int N;
int graph[MAXN][MAXN];

int minDistance(int dist[], bool visited[]) {
    int min = INF, minIndex = -1;
    for (int v = 0; v < N; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void dijkstra(int src) {
    int dist[MAXN];
    bool visited[MAXN];

    for (int i = 0; i < N; i++) {
```

```
        dist[i] = INF;
        visited[i] = false;
    }
    dist[src] = 0;

    for (int count = 0; count < N - 1; count++) {
        int u = minDistance(dist, visited);
        if (u == -1) break; // no more reachable vertices
        visited[u] = true;

        for (int v = 0; v < N; v++) {
            if (!visited[v] && graph[u][v] != 0 && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    // Print results
    printf("Vertex Distance from Source\n");
    for (int i = 0; i < N; i++) {
        printf("%d\t\t%d\n", i, dist[i]);
    }
}

int main() {
    scanf("%d", &N);

    // Read adjacency matrix
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int src;
    scanf("%d", &src);

    dijkstra(src);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

3. Problem Statement

Liam, a compiler designer, is optimizing task scheduling in a dependency-based system. He is working with a Directed Acyclic Graph (DAG), where each node represents a task, and a directed edge from one node to another signifies a dependency.

To schedule the tasks optimally, Liam must perform a topological sort using Depth-First Search (DFS), sorting the tasks based on their departure times in descending order.

Your task is to help Liam implement this topological sorting algorithm.

Input Format

The first line contains an integer N, representing the number of vertices (tasks) in the DAG.

The second line contains an integer E, representing the number of directed edges (dependencies) in the graph.

The next E lines each contain two space-separated integers:

- src The starting vertex of a directed edge.
- dest The ending vertex of a directed edge.

Output Format

The output prints N space-separated integers in a single line, representing the 0-indexed vertices in descending order of their DFS departure times, each followed by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
6
5 2
5 0
4 0
4 1
2 3
3 1
Output: 5 4 2 3 1 0

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100

int N, E;
int adj[MAXN][MAXN];
int visited[MAXN];
int topo[MAXN];
int idx = 0;

void dfs(int v) {
    visited[v] = 1;
    for (int j = 0; j < N; j++) {
        if (adj[v][j] == 1 && !visited[j]) {
            dfs(j);
        }
    }
    topo[idx++] = v; // store vertex when DFS finishes
}

int main() {
    scanf("%d", &N);
    scanf("%d", &E);

    // Initialize adjacency matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            adj[i][j] = 0;

    // Read edges
```

```
    for (int i = 0; i < E; i++) {
        int src, dest;
        scanf("%d %d", &src, &dest);
        adj[src][dest] = 1;
    }

    // Perform DFS from all vertices
    for (int i = 0; i < N; i++) visited[i] = 0;
    for (int i = 0; i < N; i++) {
        if (!visited[i]) dfs(i);
    }

    // Print topological order (reverse of finishing times)
    for (int i = idx - 1; i >= 0; i--) {
        printf("%d ", topo[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Partially correct **Marks : 1/10**

4. Problem Statement:

You are given a directed acyclic graph (DAG) G with N vertices and M edges. From G, a new graph G^ is constructed using the same vertex set and the following rules:

If there exists an edge from vertex u to vertex v in G, then there is no edge from v to u in G^.If there exists an edge from u to v in G, and from v to w in G, then there exists an edge from u to w in G^.In G^, find the maximum possible size of a set S of vertices such that for every pair of vertices (u, v) in S, there exists a directed edge between them (either u v or v u).

Example:

Input: N=3, M=2, edges: 1 2, 2 3

G^ contains edges: 1 2, 2 3, and 1 3 (by rule 2).

Set S = {1, 2, 3}: edge exists between every pair (1 2, 2 3, 1 3). This is valid

with size 3.

Output: 3

Input Format

The first line contains two space-separated integers N and M, representing the number of vertices and edges in graph G.

The next M lines each contain two space-separated integers u and v (1 ≤ u, v ≤ N), representing a directed edge from vertex u to vertex v.

Output Format

Print a single integer representing the maximum possible size of set S.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 3 2
1 2
2 3

Output: 3

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100

int N, M;
int adj[MAXN][MAXN];
int dp[MAXN];   // longest path ending at vertex
int visited[MAXN];

void dfs(int u) {
   visited[u] = 1;
   for (int v = 0; v < N; v++) {
      if (adj[u][v]) {
```

```
            if (!visited[v]) dfs(v);
            if (dp[v] < dp[u] + 1) {
                dp[v] = dp[u] + 1;
            }
        }
    }
}

int main() {
    scanf("%d %d", &N, &M);

    // Initialize adjacency matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            adj[i][j] = 0;

    // Read edges
    for (int i = 0; i < M; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        u--; v--; // convert to 0-index
        adj[u][v] = 1;
    }

    // Initialize dp
    for (int i = 0; i < N; i++) {
        dp[i] = 1; // each vertex alone is length 1
        visited[i] = 0;
    }

    // Run DFS from all vertices
    for (int i = 0; i < N; i++) {
        if (!visited[i]) dfs(i);
    }

    // Find maximum chain length
    int ans = 1;
    for (int i = 0; i < N; i++) {
        if (dp[i] > ans) ans = dp[i];
    }

    printf("%d\n", ans);
```

```
    return 0;
}
```

Status : Partially correct ***Marks : 3.5/10***

5. Problem Statement

You are given a directed acyclic graph (DAG) represented as an adjacency list.

Your task is to perform a topological sort on the graph and print the vertices in topological order. The input graph is guaranteed to be a directed acyclic graph.

Input Format

The first line of input consists of an integer N, representing the number of vertices in the directed acyclic graph.

The second line consists of an integer E, representing the number of directed edges in the graph.

The following E lines consist of two space-separated integers, src and dest, representing a directed edge from vertex src to vertex dest.

Output Format

The output consists of the vertices in topological order separated by spaces.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
6
5 2
5 0
4 0
4 1

2 3
3 1
Output: 4 5 0 2 3 1

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100

int N, E;
int adj[MAXN][MAXN];   // adjacency matrix for simplicity
int visited[MAXN];
int topo[MAXN];
int idx = 0;

void dfs(int v) {
    visited[v] = 1;
    for (int j = 0; j < N; j++) {
        if (adj[v][j] == 1 && !visited[j]) {
            dfs(j);
        }
    }
    topo[idx++] = v; // store vertex when DFS finishes
}

int main() {
    scanf("%d", &N);
    scanf("%d", &E);

    // Initialize adjacency matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            adj[i][j] = 0;

    // Read edges
    for (int i = 0; i < E; i++) {
        int src, dest;
        scanf("%d %d", &src, &dest);
        adj[src][dest] = 1;
    }
```

```
    // Perform DFS from all vertices
    for (int i = 0; i < N; i++) visited[i] = 0;
    for (int i = 0; i < N; i++) {
        if (!visited[i]) dfs(i);
    }

    // Print topological order (reverse of finishing times)
    for (int i = idx - 1; i >= 0; i--) {
        printf("%d ", topo[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Partially correct **Marks : 6.5/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Sophia, a robotics engineer, is designing a task execution system where tasks have dependencies. Each task is represented as a node in a directed acyclic graph (DAG), and a directed edge (u v) means task u must be completed before task v.

To determine the correct execution order, Sophia needs to perform a topological sort on the graph.

Your task is to help Sophia implement topological sorting and print the order in which tasks should be executed.

Input Format

The first line contains an integer N, representing the number of vertices (tasks)

in the DAG.

The second line contains an integer E, representing the number of directed edges (dependencies) in the graph.

The next E lines each contain two space-separated integers:

- src The starting vertex of a directed edge.
- dest The ending vertex of a directed edge.

Output Format

The output prints N space-separated integers in a single line, representing the vertices in topological order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
6
5 2
5 0
4 0
4 1
2 3
3 1
Output: 4 5 0 2 3 1

Answer

```
#include <stdio.h>

#define MAXN 100

int n;
int adj[MAXN][MAXN];
int visited[MAXN];
int topo[MAXN];
int idx = 0;
```

```
// Depth First Search
void dfs(int v) {
   visited[v] = 1;
   for (int j = 0; j < n; j++) {
      if (adj[v][j] == 1 && !visited[j]) {
         dfs(j);
      }
   }
   topo[idx++] = v; // store vertex when DFS finishes
}

int main() {
   scanf("%d", &n);

   // Construct adjacency matrix: lower triangular with 1s below diagonal
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         if (j < i) adj[i][j] = 1;
         else adj[i][j] = 0;
      }
   }

   // Print adjacency matrix
   printf("Adjacency Matrix of the graph\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         printf("\t%d", adj[i][j]);
      }
      printf("\n");
   }
   printf("\n");

   // Perform DFS from all vertices
   for (int i = 0; i < n; i++) visited[i] = 0;
   for (int i = 0; i < n; i++) {
      if (!visited[i]) dfs(i);
   }

   // Print topological order (reverse of finishing times)
   printf("Topological order:\n");
   for (int i = idx - 1; i >= 0; i--) {
      printf("-->%d", topo[i]);
```

```
    }
    printf("\n");

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

2. Problem Statement

Write a program to find the topological order of vertices in a directed acyclic graph (DAG) represented as an adjacency list. This involves establishing a linear arrangement of vertices such that, for every directed edge (u, v), vertex u precedes vertex v in the sequence.

Your task specifically requires printing the topological order in the reverse order of their respective end times obtained through a depth-first search (DFS) traversal.

Input Format

The first line consists of an integer N, representing the number of vertices.

The second line consists of an integer E, representing the number of directed edges.

The next E lines each contain two space-separated integers x and y, representing a directed edge from vertex x to vertex y.

Output Format

The output prints Topological Order: followed by N space-separated integers representing the vertices (0-indexed) in topological order, in reverse order of DFS end times.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
5
1 2
2 3
3 4
3 5
4 5

Output: Topological Order: 1 2 3 4 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100

int N, E;
int adj[MAXN][MAXN];   // adjacency list stored as matrix for simplicity
int visited[MAXN];
int topo[MAXN];
int idx = 0;

void dfs(int v) {
   visited[v] = 1;
   for (int j = 0; j < N; j++) {
      if (adj[v][j] == 1 && !visited[j]) {
         dfs(j);
      }
   }
   topo[idx++] = v; // store vertex when DFS finishes
}

int main() {
   scanf("%d", &N);
   scanf("%d", &E);

   // Initialize adjacency matrix
   for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
         adj[i][j] = 0;

   // Read edges
```

```
    for (int i = 0; i < E; i++) {
        int x, y;
        scanf("%d %d", &x, &y);
        adj[x][y] = 1;
    }

    // Perform DFS from all vertices
    for (int i = 0; i < N; i++) visited[i] = 0;
    for (int i = 0; i < N; i++) {
        if (!visited[i]) dfs(i);
    }

    // Print topological order (reverse of finishing times)
    printf("Topological Order: ");
    for (int i = idx - 1; i >= 0; i--) {
        printf("%d ", topo[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Wrong **Marks : 0/10**

3. Problem Statement

Given a total of N tasks labeled from 0 to N-1. Some tasks may have prerequisites. A prerequisite pair [u, v] means task u must be completed before task v. Given the total number of tasks and a list of prerequisite pairs, return the ordering of tasks you should complete to finish all tasks. There may be multiple correct orders — return any one of them. If it is impossible to finish all tasks (due to a cycle), print nothing.

Examples

Input: N=2, Prerequisite_Pairs=[[1, 0]]

Output: [0, 1]

Explanation: There are a total of 2 tasks to pick. To pick task 1 you should have finished task 0. So the correct task order is [0, 1] .

Input: N=4, Prerequisite_Pairs=[[1, 0], [2, 0], [3, 1], [3, 2]]

Output: [0, 1, 2, 3] or [0, 2, 1, 3]

Explanation: There are a total of 4 tasks to pick. To pick task 3 you should have finished both tasks 1 and 2. Both tasks 1 and 2 should be pick after you finished task 0. So one correct task order is [0, 1, 2, 3]. Another correct ordering is [0, 2, 1, 3].

Input Format

The first line of input consists of an integer n, representing the number of tasks.

The second line consists of an integer p, representing the number of prerequisite pairs.

The next p lines each consist of two space-separated integers u and v, representing that task u must be completed before task v.

Output Format

The output prints the n tasks as space-separated integers on a single line, representing a valid task execution order. If no valid ordering exists, print nothing.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
0

Output: 0 1 2

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100

int main() {
    int N, P;
```

```
scanf("%d", &N);
scanf("%d", &P);

int adj[MAXN][MAXN];   // adjacency matrix for simplicity
int indegree[MAXN];
for (int i = 0; i < N; i++) {
    indegree[i] = 0;
    for (int j = 0; j < N; j++) {
        adj[i][j] = 0;
    }
}

// Read prerequisite pairs
for (int i = 0; i < P; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    if (!adj[u][v]) { // avoid duplicate edges
        adj[u][v] = 1;
        indegree[v]++;
    }
}

// Queue for tasks with indegree 0
int queue[MAXN], front = 0, rear = 0;
for (int i = 0; i < N; i++) {
    if (indegree[i] == 0) {
        queue[rear++] = i;
    }
}

int count = 0;
int order[MAXN];

while (front < rear) {
    int u = queue[front++];
    order[count++] = u;

    for (int v = 0; v < N; v++) {
        if (adj[u][v]) {
            indegree[v]--;
            if (indegree[v] == 0) {
                queue[rear++] = v;
```

```
            }
        }
    }
}

// If cycle exists, count < N
if (count < N) {
    // Print nothing
    return 0;
}

// Print valid order
for (int i = 0; i < count; i++) {
    printf("%d ", order[i]);
}
printf("\n");

return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Raj is given a weighted directed graph with N vertices and the weights of the edges between the vertices.

His task is to implement Dijkstra's algorithm to find the shortest distance from a given source vertex to all other vertices in the graph.

Input Format

The first line contains an integer N, the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at the i-th row and j-th column represents the weight of the edge from vertex i to vertex j. A value of 0 indicates no direct edge between the vertices.

The last line contains an integer src, representing the source vertex.

Output Format

The first line of output prints the heading: Vertex Distance from Source

The next N lines each print the vertex number and its shortest distance from the source vertex, separated by two tab characters. Vertices are numbered from 0 to N-1.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 4
0 2 0 5
2 0 4 0
0 4 0 1
5 0 1 0
0
Output: Vertex Distance from Source
0 0
1 2
2 6
3 5

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define MAXN 100
#define INF INT_MAX

int N;
int graph[MAXN][MAXN];

int minDistance(int dist[], bool visited[]) {
    int min = INF, minIndex = -1;
    for (int v = 0; v < N; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
```

```
            minIndex = v;
        }
    }
    return minIndex;
}

void dijkstra(int src) {
    int dist[MAXN];
    bool visited[MAXN];

    for (int i = 0; i < N; i++) {
        dist[i] = INF;
        visited[i] = false;
    }
    dist[src] = 0;

    for (int count = 0; count < N - 1; count++) {
        int u = minDistance(dist, visited);
        if (u == -1) break; // no more reachable vertices
        visited[u] = true;

        for (int v = 0; v < N; v++) {
            if (!visited[v] && graph[u][v] != 0 && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    // Print results
    printf("Vertex Distance from Source\n");
    for (int i = 0; i < N; i++) {
        printf("%d\t\t%d\n", i, dist[i]);
    }
}

int main() {
    scanf("%d", &N);

    // Read adjacency matrix
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
```

```
            scanf("%d", &graph[i][j]);
        }
    }

    int src;
    scanf("%d", &src);

    dijkstra(src);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Anu is given a weighted, directed, and connected graph with V vertices and E edges represented as an adjacency list. They are also given a source vertex S.

The task is to find the shortest distance from the source vertex S to all other vertices in the graph using Dijkstra's algorithm.

Input Format

The first line contains an integer V representing the number of vertices in the graph.

The second line contains an integer E representing the number of edges in the graph.

The next E lines contain three integers each: from, to, and weight, denoting an edge between vertices from and to with the given weight.

The last line contains an integer S representing the source vertex.

Output Format

The output prints a single line in the format:

Shortest distances from vertex S to all other vertices: d0 d1 d2 ... dV-1

where S is the source vertex number and d0, d1, ..., dV-1 are the shortest distances from S to vertices 0 through V-1 respectively, each followed by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
1
0 1 9
0
Output: Shortest distances from vertex 0 to all other vertices: 0 9

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define MAXN 100
#define INF INT_MAX

int N;
int graph[MAXN][MAXN];

int minDistance(int dist[], bool visited[]) {
    int min = INF, minIndex = -1;
    for (int v = 0; v < N; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void dijkstra(int src) {
    int dist[MAXN];
    bool visited[MAXN];
```

```
    for (int i = 0; i < N; i++) {
        dist[i] = INF;
        visited[i] = false;
    }
    dist[src] = 0;

    for (int count = 0; count < N - 1; count++) {
        int u = minDistance(dist, visited);
        if (u == -1) break; // no more reachable vertices
        visited[u] = true;

        for (int v = 0; v < N; v++) {
            if (!visited[v] && graph[u][v] != 0 && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    // Print results
    printf("Vertex Distance from Source\n");
    for (int i = 0; i < N; i++) {
        printf("%d\t\t%d\n", i, dist[i]);
    }
}

int main() {
    scanf("%d", &N);

    // Read adjacency matrix
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int src;
    scanf("%d", &src);

    dijkstra(src);

    return 0;
```

}

Status : Wrong **Marks : 0/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Ram is tasked with finding the shortest distances from a source router to all other routers in a network. The network is represented as a graph of N routers connected by M bidirectional weighted links.

The program uses Dijkstra's Algorithm: starting from the source router, it repeatedly selects the unvisited router with the smallest known distance, then updates distances to its neighbors if a shorter path is found through it. This continues until all routers have been visited.

Given the network details and a source router, find and print the shortest distance from the source router to every router in the network.

Input Format

The first line of input consists of an integer N, representing the number of routers.

The second line of input consists of an integer M, representing the number of links.

The next M lines each consist of three space-separated integers: r1, r2, and w, representing a bidirectional link between router r1 and router r2 with weight w.

The last line of input consists of an integer s, representing the source router.

Output Format

The output consists of N lines printed in ascending order of router index (from 0 to N-1).

Each line prints two space-separated integers: the router index and its shortest distance from the source router.

The source router itself is printed with distance 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
5
0 1 1
0 2 3
1 2 1
1 3 4
2 3 1
0
Output: 0 0
1 1
2 2
3 3

Answer

```
#include <stdio.h>
#include <limits.h>
```

```c
#include <stdbool.h>

#define MAXN 8
#define INF INT_MAX

int N, M;
int graph[MAXN][MAXN];

int minDistance(int dist[], bool visited[]) {
    int min = INF, minIndex = -1;
    for (int v = 0; v < N; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void dijkstra(int src) {
    int dist[MAXN];
    bool visited[MAXN];

    for (int i = 0; i < N; i++) {
        dist[i] = INF;
        visited[i] = false;
    }
    dist[src] = 0;

    for (int count = 0; count < N - 1; count++) {
        int u = minDistance(dist, visited);
        if (u == -1) break; // no reachable vertex left
        visited[u] = true;

        for (int v = 0; v < N; v++) {
            if (!visited[v] && graph[u][v] != INF && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
```

```
    // Print results
    for (int i = 0; i < N; i++) {
        if (dist[i] == INF)
            printf("%d -1\n", i); // unreachable
        else
            printf("%d %d\n", i, dist[i]);
    }
}

int main() {
    int numRouters, numLinks;
    scanf("%d", &numRouters);
    scanf("%d", &numLinks);
    int graph[MAX_ROUTERS][MAX_ROUTERS] = {0};
    int adj[MAX_ROUTERS][MAX_ROUTERS] = {0};
    for (int i = 0; i < numLinks; i++) {
        int router1, router2, weight;
        scanf("%d %d %d", &router1, &router2, &weight);
        graph[router1][router2] = weight;
        graph[router2][router1] = weight;
        adj[router1][router2] = 1;
        adj[router2][router1] = 1;
    }
    int source;
    scanf("%d", &source);
    dijkstra(graph, adj, source, numRouters);
    return 0;
}
```

Status : Wrong ***Marks : 0/10***

2. Problem Statement

Alice is working on a delivery system for an e-commerce platform. The system uses a network of delivery hubs connected by roads, each with a certain travel time in minutes.

Alice needs to find the fastest delivery time from a source hub to every other hub in the network. Given the network details and a source hub, the program uses Dijkstra's Algorithm to find and print the shortest delivery time from the source hub to all hubs in the network.

Dijkstra's Algorithm works by starting from the source hub with distance 0, then repeatedly selecting the unvisited hub with the smallest known time and updating the times to its neighbors if a shorter path is found through it. This continues until all hubs have been visited.

Input Format

The first line of input consists of an integer N, representing the number of delivery hubs.

The second line of input consists of an integer M, representing the number of roads between hubs.

The next M lines each consist of three space-separated integers: r1, r2, and t, representing a bidirectional road between hub r1 and hub r2 with a delivery time of t minutes.

The last line of input consists of an integer s, representing the source delivery hub.

Output Format

The output prints N lines representing the shortest delivery times from the source hub to all hubs (including the source) in ascending order of hub index.

Each line prints two space-separated integers: the hub index and its shortest delivery time from the source hub.

The delivery time of the source hub itself is 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
3
0 1 2
0 2 4
1 2 1
0
Output: 0 0

1 2
2 3

Answer

```
#include <stdio.h>
#include <limits.h>

#define MAX_NODES 10



#define MAXN 10
#define INF INT_MAX

int N, M;
int graph[MAXN][MAXN];

int minDistance(int dist[], bool visited[]) {
    int min = INF, minIndex = -1;
    for (int v = 0; v < N; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void dijkstra(int src) {
    int dist[MAXN];
    bool visited[MAXN];

    for (int i = 0; i < N; i++) {
        dist[i] = INF;
        visited[i] = false;
    }
    dist[src] = 0;

    for (int count = 0; count < N - 1; count++) {
        int u = minDistance(dist, visited);
        if (u == -1) break; // no more reachable hubs
        visited[u] = true;
```

```
            for (int v = 0; v < N; v++) {
                if (!visited[v] && graph[u][v] != INF && dist[u] != INF
                    && dist[u] + graph[u][v] < dist[v]) {
                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }

        // Print shortest delivery times
        for (int i = 0; i < N; i++) {
            if (dist[i] == INF)
                printf("%d -1\n", i); // unreachable hub
            else
                printf("%d %d\n", i, dist[i]);
        }
    }

    int main() {
        int N, M, s;
        scanf("%d", &N);
        scanf("%d", &M);
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                graph[i][j] = -1;
            }
        }
        for (int i = 0; i < M; i++) {
            int r1, r2, w;
            scanf("%d %d %d", &r1, &r2, &w);
            graph[r1][r2] = w;
            graph[r2][r1] = w;
        }
        scanf("%d", &s);
        dijkstra(N, s);
        for (int i = 0; i < N; i++) {
            printf("%d %d\n", i, dist[i]);
        }
        return 0;
    }
```

Status : Wrong **Marks : 0/10**

3. Problem statement

Create a program to calculate and display the topological order of a Directed Acyclic Graph (DAG) with n vertices. The graph is represented as a lower triangular adjacency matrix, where for every vertex i and vertex j with j < i, the edge from i to j exists (value 1), and no edge exists from j to i (value 0). All diagonal and upper triangular values are 0.

The program should:

Construct and display the adjacency matrix of the graph.Perform a Depth First Search (DFS) traversal on the graph.Display the topological order of the vertices in reverse order of their DFS end times.

Input Format

The first line of input consists of an integer n, representing the number of vertices in the graph. No further input is required; the adjacency matrix is constructed internally by the program.

Output Format

he first line of output prints: Adjacency Matrix of the graph

The next n lines print the adjacency matrix row by row, where each value is preceded by a tab character (\t).

A blank line is printed after the matrix.

The next line prints: Topological order:

The final line prints the topological order of the vertices in the format: -->v1-->v2-->...-->vn, where vertices appear in reverse order of their DFS end times.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
Output: Adjacency Matrix of the graph
	0	0	0
	1	0	0

1 1 0

Topological order:
-->2-->1-->0

Answer

```
#include<stdio.h>

int s[100], res[100], top;
#include <stdio.h>

#define MAXN 100

int n;
int adj[MAXN][MAXN];
int visited[MAXN];
int topo[MAXN];
int idx = 0;

void dfs(int v) {
    visited[v] = 1;
    for (int j = 0; j < n; j++) {
        if (adj[v][j] == 1 && !visited[j]) {
            dfs(j);
        }
    }
    topo[idx++] = v; // store vertex when DFS finishes
}

int main() {
    scanf("%d", &n);

    // Construct adjacency matrix: lower triangular with 1s below diagonal
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (j < i) adj[i][j] = 1;
            else adj[i][j] = 0;
        }
    }

    // Print adjacency matrix
    printf("Adjacency Matrix of the graph\n");
```

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("\t%d", adj[i][j]);
        }
        printf("\n");
    }
    printf("\n");

    // Perform DFS from all vertices
    for (int i = 0; i < n; i++) visited[i] = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) dfs(i);
    }

    // Print topological order (reverse of finishing times)
    printf("Topological order:\n");
    for (int i = idx - 1; i >= 0; i--) {
        printf("-->%d", topo[i]);
    }
    printf("\n");

    return 0;
}


int main() {
    int a[100][100], n, i, j;
    scanf("%d", &n);
    buildMatrix(a, n);
    printf("Adjacency Matrix of the graph\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++)
            printf("\t%d", a[i][j]);
        printf("\n");
    }
    printf("\nTopological order:\n");
    topological_order(n, a);
    for (i = top - 1; i >= 0; i--)
        printf("-->%d", res[i]);
    printf("\n");
    return 0;
}
```

Status : Wrong **Marks : 0/10**

4. Problem Statement

Write a program to implement and print the topological order of a directed acyclic graph (DAG). The program should take the number of vertices and the adjacency matrix of the graph as input and then output the topological order.

When multiple vertices have zero indegree simultaneously, process them in ascending order of their vertex number (1-indexed). The output vertices are numbered starting from 1

Input Format

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines contain the adjacency matrix of the graph. Each line contains N space-separated integers (0 or 1), where 1 indicates a directed edge from the corresponding row vertex to the column vertex.

Output Format

The output prints the topological order of the vertices as space-separated 1-indexed integers on a single line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
1 0 0
0 1 0
1 0 0

Output: 3 1 2

Answer

```
#include <stdio.h>
```

```
#define MAXN 100

int n;
int adj[MAXN][MAXN];
int visited[MAXN];
int topo[MAXN];
int idx = 0;

// Depth First Search
void dfs(int v) {
   visited[v] = 1;
   for (int j = 0; j < n; j++) {
      if (adj[v][j] == 1 && !visited[j]) {
         dfs(j);
      }
   }
   topo[idx++] = v; // store vertex when DFS finishes
}

int main() {
   scanf("%d", &n);

   // Construct adjacency matrix: lower triangular with 1s below diagonal
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         if (j < i) adj[i][j] = 1;
         else adj[i][j] = 0;
      }
   }

   // Print adjacency matrix
   printf("Adjacency Matrix of the graph\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         printf("\t%d", adj[i][j]);
      }
      printf("\n");
   }
   printf("\n");

   // Perform DFS from all vertices
```

```
    for (int i = 0; i < n; i++) visited[i] = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) dfs(i);
    }

    // Print topological order (reverse of finishing times)
    printf("Topological order:\n");
    for (int i = idx - 1; i >= 0; i--) {
        printf("-->%d", topo[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Wrong **Marks : 0/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 12.5

Section 1 : MCQ

1. Which type of graphs can be topologically sorted?

Answer

Directed graphs

Status : Correct ***Marks : 1/1***

2. Topological sort can be used to detect which of the following?

Answer

Cycles in a directed graph

Status : Correct ***Marks : 1/1***

3. In topological sort, vertices are visited in which order?

Answer

In topological order

Status : Correct ***Marks : 1/1***

4. Which of the following data structures can be used to implement topological sort?

Answer

All of the listed options

Status : Wrong ***Marks : 0/1***

5. Which of the following algorithms can be used to perform topological sort?

Answer

Depth First Search (DFS)

Status : Partially correct ***Marks : 0.5/1***

6. Topological sort is equivalent to which of the traversals in trees?

Answer

Post-order traversal

Status : Wrong ***Marks : 0/1***

7. When is the topological sort of a graph unique?

Answer

When there exists a hamiltonian path in the graph

Status : Correct ***Marks : 1/1***

8. In Dijkstra's algorithm, if two vertices u and v have the same tentative distance from the source, which one is chosen first?

Answer

It can be arbitrary

Status : Correct ***Marks : 1/1***

9. In a graph with non-negative weights, why is Dijkstra's algorithm guaranteed to find the shortest path?

Answer

Because once a vertex's distance is finalized, it cannot be improved

Status : Correct ***Marks : 1/1***

10. How can Dijkstra's algorithm be adapted to return not only distances but also the actual paths?

Answer

Maintain a parent array for each vertex

Status : Correct ***Marks : 1/1***

11. In Dijkstra's algorithm, what is the purpose of the relaxation step when processing an edge (u, v)?

Answer

To update the distance to vertex v if a shorter path through u is found

Status : Correct ***Marks : 1/1***

12. In a graph with multiple edges between the same pair of vertices, how should Dijkstra's algorithm handle them?

Answer

Consider only the edge with the minimum weight

Status : Correct **Marks : 1/1**

13. If a graph has an edge weight of zero, will Dijkstra's algorithm still work correctly?

Answer

Yes, always

Status : Correct **Marks : 1/1**

14. Which property of Dijkstra's algorithm guarantees that the shortest path to a vertex is correct when the vertex is added to the finalized set?

Answer

Both A and B

Status : Correct **Marks : 1/1**

15. Why can Dijkstra's algorithm fail on a graph with a negative weight edge even if there are no negative cycles?

Answer

Because a previously finalized vertex may get a shorter path later

Status : Correct **Marks : 1/1**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Mary is planning to build a railway network to connect cities efficiently. She needs a program that, given the number of cities and the segments between them, determines the minimum cost to connect all cities using an undirected weighted graph.

Implement a solution using Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the city network. If it is not possible to connect all cities, output an appropriate message

Example

Input

3

3

0 1 4

1 2 5

0 2 3

Output

7

Explanation

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.

Track Segment 2: Connects city 1 and city 2 with a weight of 5.

Track Segment 3: Connects city 0 and city 2 with a weight of 3.

The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is 3 + 4 = 7.

Input Format

The first line of input contains an integer num_cities, representing the number of cities (numbered 0 to num_cities - 1).

The second line contains an integer num_segments, representing the number of undirected segments (edges) between cities.

The next num_segments lines each contain three space-separated integers: source, destination, and weight, representing an undirected segment between city source and city destination with the given cost.

Output Format

The first line of output prints an integer representing the minimum cost to connect all cities.

If it is not possible to connect all the cities, the output prints 'It is not possible to connect all cities' (without a period).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
3
0 1 4
1 2 5
0 2 3
Output: 7

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Structure to represent an edge
typedef struct {
    int u, v, w;
} Edge;

// Compare function for qsort
int compare(const void* a, const void* b) {
    return ((Edge*)a)->w - ((Edge*)b)->w;
}

// Find function for Union-Find
int find(int parent[], int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent, parent[i]);
}

// Union function for Union-Find
void unionSet(int parent[], int rank[], int x, int y) {
    int xroot = find(parent, x);
    int yroot = find(parent, y);

    if (rank[xroot] < rank[yroot]) parent[xroot] = yroot;
```

```
    else if (rank[xroot] > rank[yroot]) parent[yroot] = xroot;
    else {
        parent[yroot] = xroot;
        rank[xroot]++;
    }
}

int main() {
    int num_cities, num_segments;
    scanf("%d", &num_cities);
    scanf("%d", &num_segments);

    Edge edges[num_segments];
    for (int i = 0; i < num_segments; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    // Sort edges by weight
    qsort(edges, num_segments, sizeof(Edge), compare);

    int parent[num_cities], rank[num_cities];
    for (int i = 0; i < num_cities; i++) {
        parent[i] = i;
        rank[i] = 0;
    }

    int totalCost = 0;
    int edgeCount = 0;

    for (int i = 0; i < num_segments && edgeCount < num_cities - 1; i++) {
        int u = edges[i].u;
        int v = edges[i].v;
        int w = edges[i].w;

        int setU = find(parent, u);
        int setV = find(parent, v);

        if (setU != setV) {
            totalCost += w;
            edgeCount++;
            unionSet(parent, rank, setU, setV);
        }
```

```
    }
    if (edgeCount == num_cities - 1)
        printf("%d\n", totalCost);
    else
        printf("It is not possible to connect all cities\n");

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Maya, a project planner, must connect multiple office branches using network cables while keeping the total cost within a given budget. Each possible connection between two branches has an associated cost.

Write a program to find the Minimum Spanning Tree (MST) of the branch network using Kruskal's Algorithm on an undirected weighted graph. If the total cost of the MST is within the given budget, display the cost. Otherwise, indicate that it is not possible.

It is guaranteed that the graph is connected in all test cases

Input Format

The first line contains three space-separated integers V, E, and budget, representing the number of branches (vertices, numbered 1 to V), the number of possible connections (edges), and the total budget respectively.

Each of the next E lines contains three space-separated integers src, dest, and weight, representing an undirected connection between branch src and branch dest with cost weight.

Output Format

If the MST cost is within the budget, the output prints:

'Minimum Cost Spanning Tree within the budget: X'

where X is an integer representing the total minimum cost.

Otherwise, the output prints:

'A valid MST within the budget is not possible.'

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 3 5
1 2 2
2 3 3
3 1 4
Output: Minimum Cost Spanning Tree within the budget: 5

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Structure to represent an edge
typedef struct {
    int src, dest, weight;
} Edge;

// Compare function for qsort
int compare(const void* a, const void* b) {
    return ((Edge*)a)->weight - ((Edge*)b)->weight;
}

// Find function for Union-Find
int find(int parent[], int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent, parent[i]);
}

// Union function for Union-Find
void unionSet(int parent[], int rank[], int x, int y) {
    int xroot = find(parent, x);
    int yroot = find(parent, y);
```

```
    if (rank[xroot] < rank[yroot]) parent[xroot] = yroot;
    else if (rank[xroot] > rank[yroot]) parent[yroot] = xroot;
    else {
        parent[yroot] = xroot;
        rank[xroot]++;
    }
}

int main() {
    int V, E;
    long long budget;
    scanf("%d %d %lld", &V, &E, &budget);

    Edge edges[E];
    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].src, &edges[i].dest, &edges[i].weight);
        edges[i].src--;   // Convert to 0-index
        edges[i].dest--;
    }

    // Sort edges by weight
    qsort(edges, E, sizeof(Edge), compare);

    int parent[V], rank[V];
    for (int i = 0; i < V; i++) {
        parent[i] = i;
        rank[i] = 0;
    }

    long long totalCost = 0;
    int edgeCount = 0;

    for (int i = 0; i < E && edgeCount < V - 1; i++) {
        int u = edges[i].src;
        int v = edges[i].dest;
        int w = edges[i].weight;

        int setU = find(parent, u);
        int setV = find(parent, v);

        if (setU != setV) {
```

```
            totalCost += w;
            edgeCount++;
            unionSet(parent, rank, setU, setV);
        }
    }

    if (edgeCount == V - 1 && totalCost <= budget)
        printf("Minimum Cost Spanning Tree within the budget: %lld\n", totalCost);
    else
        printf("A valid MST within the budget is not possible.\n");

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Sita is planning to connect multiple locations in her city with roads. Each possible road between two locations has a construction distance (in kilometres). The city is represented as an undirected weighted graph using an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to numLocations - 1).

To minimise the overall construction distance, she decides to use Prim's Algorithm to construct the Minimum Spanning Tree (MST). Help her find the MST edges and the total minimum distance.

Input Format

The first line of input contains an integer numLocations, representing the number of locations (numbered 0 to numLocations - 1).

The next numLocations lines each contain numLocations space-separated integers representing the adjacency matrix, where graph[i][j] is the distance between location i and location j. A value of 0 indicates no direct road between those two locations.

Output Format

The first line of output prints: 'Location\tDistance'

Each of the next numLocations - 1 lines prints the MST edge in the format:

'parent -> child \t\t\t dist km'

where parent is the parent location number, child is the child location number, and dist is the edge weight (integer) in kilometres.

The last line of output prints: 'Minimum total distance: totalCost km'

where totalCost is an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2
0 5
5 0
Output: Location Distance
0 -> 1   5 km
Minimum total distance: 5 km

Answer

```
#include <stdio.h>
#include <limits.h>

#define MAX 10

// Function to find the vertex with minimum key value not yet included in MST
int minKey(int key[], int mstSet[], int n) {
   int min = INT_MAX, min_index = -1;
   for (int v = 0; v < n; v++) {
      if (mstSet[v] == 0 && key[v] < min) {
         min = key[v];
         min_index = v;
      }
```

```
    }
    return min_index;
}

// Prim's Algorithm
void primMST(int graph[MAX][MAX], int n) {
    int parent[n];   // Stores MST edges
    int key[n];      // Minimum edge weight to connect each vertex
    int mstSet[n];   // Tracks vertices included in MST

    // Initialize all keys as infinite
    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    // Start from vertex 0
    key[0] = 0;
    parent[0] = -1;

    // Build MST with n-1 edges
    for (int count = 0; count < n - 1; count++) {
        int u = minKey(key, mstSet, n);
        mstSet[u] = 1;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    // Print MST edges and total cost
    int totalCost = 0;
    printf("Location\tDistance\n");
    for (int i = 1; i < n; i++) {
        printf("%d -> %d\t\t\t %d km\n", parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }
    printf("Minimum total distance: %d km\n", totalCost);
}
```

```
int main() {
    int numLocations;
    scanf("%d", &numLocations);

    int graph[MAX][MAX];
    for (int i = 0; i < numLocations; i++) {
        for (int j = 0; j < numLocations; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    primMST(graph, numLocations);
    return 0;
}
```

Status : Wrong **Marks : 0/10**

4. Problem Statement

Riya works at a data tools company that needs a compact representation of frequently used characters for fast transmission. She decides to use Huffman Coding, a lossless compression technique that assigns shorter bit-strings to more frequent symbols and longer bit-strings to less frequent ones.

The Huffman codes should be printed in the order that leaf nodes are encountered during a left-to-right traversal of the constructed Huffman tree.

Write a program that builds a Huffman tree from a list of symbols with their frequencies and prints the Huffman codes for each symbol.

Input Format

The first line contains an integer n, representing the number of distinct symbols.

Each of the next n lines contains a single printable character (the symbol) followed by a space and a positive integer (the frequency), representing the symbol and its occurrence count.

Output Format

The output prints n lines, one per symbol, in the order that leaf nodes are encountered during a left-to-right traversal of the Huffman tree (left child before right child at every internal node).

Each line prints the symbol followed by a colon, a space, and its Huffman code (a binary string of 0s and 1s):

'<symbol>: <binary_code>'

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2
a 1
b 2

Output: a: 0
b: 1

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 256

// Huffman tree node
struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
};

// MinHeap structure
struct MinHeap {
```

```
    int size;
    int capacity;
    struct Node** array;
};

// Create new node
struct Node* newNode(char ch, int freq) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->ch = ch;
    temp->freq = freq;
    temp->left = temp->right = NULL;
    return temp;
}

// Create min heap
struct MinHeap* createMinHeap(int capacity) {
    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct MinHeap));
    minHeap->size = 0;
    minHeap->capacity = capacity;
    minHeap->array = (struct Node**)malloc(capacity * sizeof(struct Node*));
    return minHeap;
}

// Swap nodes
void swapNode(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}

// Heapify
void minHeapify(struct MinHeap* minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < minHeap->size && minHeap->array[left]->freq < minHeap-
>array[smallest]->freq)
        smallest = left;

    if (right < minHeap->size && minHeap->array[right]->freq < minHeap-
>array[smallest]->freq)
```

```
    smallest = right;

  if (smallest != idx) {
    swapNode(&minHeap->array[smallest], &minHeap->array[idx]);
    minHeapify(minHeap, smallest);
  }
}

// Extract min
struct Node* extractMin(struct MinHeap* minHeap) {
  struct Node* temp = minHeap->array[0];
  minHeap->array[0] = minHeap->array[minHeap->size - 1];
  minHeap->size--;
  minHeapify(minHeap, 0);
  return temp;
}

// Insert node
void insertMinHeap(struct MinHeap* minHeap, struct Node* node) {
  minHeap->size++;
  int i = minHeap->size - 1;
  while (i && node->freq < minHeap->array[(i - 1) / 2]->freq) {
    minHeap->array[i] = minHeap->array[(i - 1) / 2];
    i = (i - 1) / 2;
  }
  minHeap->array[i] = node;
}

// Build min heap
void buildMinHeap(struct MinHeap* minHeap) {
  int n = minHeap->size - 1;
  for (int i = (n - 1) / 2; i >= 0; i--)
    minHeapify(minHeap, i);
}

// Create and build min heap
struct MinHeap* createAndBuildMinHeap(char data[], int freq[], int size) {
  struct MinHeap* minHeap = createMinHeap(size);
  for (int i = 0; i < size; i++)
    minHeap->array[i] = newNode(data[i], freq[i]);
  minHeap->size = size;
  buildMinHeap(minHeap);
```

```
    return minHeap;
}

// Build Huffman tree
struct Node* buildHuffmanTree(char data[], int freq[], int
```

Status : Wrong **Marks : 0/10**

5. Problem Statement

In a data science class, Professor Huffman demonstrates how encoded messages can be decoded using a pre-built Huffman tree.

Given a Huffman code table (mapping each character to its binary code) and an encoded binary message, decode the message back to its original text by traversing the Huffman tree.

The code table is guaranteed to be a valid prefix-free Huffman code.

The encoded message consists only of '0' and '1' characters, and every bit maps to exactly one symbol in the Huffman tree

The tree is defined as follows:

Internal nodes are represented by "*".The left edge (0) leads to the symbol "A".The right edge (1) leads to the symbol "B".

This means:

"0" A"1" B

Your task is to decode a given encoded message (a string of 0s and 1s) back into its original form consisting of "A" and "B".

Input Format

The first line contains an integer n, representing the number of symbols in the code table.

Each of the next n lines contains a character and its Huffman code (binary string), separated by a space.

The last line contains the encoded binary message (a string of 0s and 1s) to be

decoded

Output Format

The output prints the decoded message as a sequence of characters ('A' or 'B'), followed by a newline.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 11100011100

Output: BBBAAABBBAA

Answer

```
#include <stdio.h>
#include <string.h>

int main() {
    int n;
    scanf("%d", &n);  // number of symbols in the code table

    // Read the code table (not really needed here, but included for format
compliance)
    char symbol;
    char code[101];
    for (int i = 0; i < n; i++) {
        scanf(" %c %s", &symbol, code);
    }

    // Read the encoded message
    char encoded[101];
    scanf("%s", encoded);

    // Decode: 0 -> A, 1 -> B
    for (int i = 0; i < strlen(encoded); i++) {
        if (encoded[i] == '0')
            printf("A");
        else if (encoded[i] == '1')
            printf("B");
    }
```

```
    printf("\n");

    return 0;
}
```

Status : Wrong *Marks : 0/10*

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 35

Section 1 : Coding

1. Problem Statement

Bob is working on a project involving network optimization. He needs a program to find the Minimum Spanning Tree (MST) of a given undirected weighted graph with 5 nodes (numbered 0 to 4).

The graph is represented as a 5×5 symmetric adjacency matrix, where a value of 0 indicates no direct connection between two nodes. Implement Prim's Algorithm to find the MST and print each edge with its weight.

Example

Input

0 3 0 5 0

4 0 2 3 5

0 3 0 8 7

3 8 0 0 9

0 5 7 2 0

Output

Edge Weight

0 - 1 4

1 - 2 3

1 - 3 8

1 - 4 5

Explanation

In the given 5x5 matrix representing an undirected graph, Bob has specified the weights of edges. The Prim's algorithm is applied to find the Minimum Spanning Tree (MST). The output displays the edges along with their corresponding weights, forming the optimal tree connecting all vertices. In this case, the MST includes edges (0-1), (1-2), (1-3), and (1-4) with weights 4, 3, 8, and 5 respectively, ensuring minimal total weight for network optimization.

Input Format

The input consists of 5 lines, each containing 5 space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two nodes. The matrix is symmetric (graph[i][j] = graph[j][i]) since the graph is undirected.

Output Format

The first line of output prints the header: 'Edge \tWeight'

Each of the next 4 lines prints one MST edge in the format:

'u - v \tw'

where u is the parent node, v is the child node, and w is the integer edge weight, printed in ascending order of child node index (node-index order).

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0 2 0 6 0
2 0 3 8 5
0 3 0 8 7
6 8 0 0 9
0 5 7 9 0

Output: Edge Weight
0 - 1 2
1 - 2 3
0 - 3 6
1 - 4 5

Answer

```
#include <stdio.h>
#include <limits.h>

#define V 5   // Fixed number of nodes

// Function to find the vertex with minimum key value not yet included in MST
int minKey(int key[], int mstSet[]) {
    int min = INT_MAX, min_index = -1;
    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}
```

```
// Prim's Algorithm
void primMST(int graph[V][V]) {
    int parent[V];   // Stores MST edges
    int key[V];      // Minimum edge weight to connect each vertex
    int mstSet[V];   // Tracks vertices included in MST

    // Initialize all keys as infinite
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    // Start from vertex 0
    key[0] = 0;
    parent[0] = -1;

    // Build MST with V-1 edges
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    // Print MST edges in ascending order of child node index
    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
    }
}

int main() {
    int graph[V][V];
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
```

```
    }

    primMST(graph);
    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

You are given a set of cities in a country and the cost of building railway track segments to connect them. Your task is to find the minimum cost required to build the railway tracks and connect all the cities. The cost of each track segment is given, along with the source city and destination city to which it connects.

Write a program that takes the number of cities, the number of track segments, and the details of each track segment as input. Implement Kruskal's Algorithm to calculate the minimum cost of building the railway tracks.

Cities are numbered 0 to N-1.

Example

Input:

3

3

0 1 4

1 2 5

0 2 3

Output:

7

Explanation:

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.Track Segment 2: Connects city 1 and city 2 with a weight of 5.Track Segment 3: Connects city 0 and city 2 with a weight of 3.The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is 3 + 4 = 7.

Input Format

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

Output Format

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
5
0 1 1

0 2 4
1 2 2
1 3 5
2 3 3

Output: 6

Answer

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

// Compare function for qsort
int compare(const void* a, const void* b) {
    return ((Edge*)a)->w - ((Edge*)b)->w;
}

// Find function for Union-Find
int find(int parent[], int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent, parent[i]);
}

// Union function for Union-Find
void unionSet(int parent[], int rank[], int x, int y) {
    int xroot = find(parent, x);
    int yroot = find(parent, y);

    if (rank[xroot] < rank[yroot]) parent[xroot] = yroot;
    else if (rank[xroot] > rank[yroot]) parent[yroot] = xroot;
    else {
        parent[yroot] = xroot;
        rank[xroot]++;
    }
}

int main() {
    int N, M;
    scanf("%d", &N);
    scanf("%d", &M);
```

```
    Edge edges[M];
    for (int i = 0; i < M; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    // Sort edges by weight
    qsort(edges, M, sizeof(Edge), compare);

    int parent[N], rank[N];
    for (int i = 0; i < N; i++) {
        parent[i] = i;
        rank[i] = 0;
    }

    int totalCost = 0;
    int edgeCount = 0;

    for (int i = 0; i < M && edgeCount < N - 1; i++) {
        int u = edges[i].u;
        int v = edges[i].v;
        int w = edges[i].w;

        int setU = find(parent, u);
        int setV = find(parent, v);

        if (setU != setV) {
            totalCost += w;
            edgeCount++;
            unionSet(parent, rank, setU, setV);
        }
    }

    if (edgeCount == N - 1)
        printf("%d\n", totalCost);
    else
        printf("It is not possible to connect all cities\n");

    return 0;
}
```

Status : Partially correct ***Marks : 7.5/10***

3. Problem Statement

Raja is given a map of cities connected by bidirectional roads, where each road has an associated construction cost (weight). His task is to find the Minimum Spanning Tree (MST) of this network using Prim's Algorithm, starting from city 0, so that all cities are connected with the minimum total construction cost.

Prim's Algorithm works as follows:

Start from the source city (city 0). Mark it as included in the MST.At each step, select the edge with the minimum weight that connects a city already in the MST to a city not yet in the MST.Repeat until all cities are included in the MST.

The program should display each MST edge (the city included and the cost to connect it) and the total MST cost.

Note: The given graph is a connected, undirected, weighted graph. Self-loops and parallel edges are not present.

Input Format

The first line of input consists of two space-separated integers, n and m, where n represents the number of cities (vertices) and m represents the number of roads (edges).

The next m lines each consist of three space-separated integers u, v, and w, where u and v are the two cities connected by the road (0-indexed) and w is the construction cost (weight) of the road.

Output Format

The output consists of n lines.

The first n-1 lines each print the MST edge added, in the format:

"Edge: u - v, Cost: w", where u is the city already in the MST, v is the newly added city, and w is the integer edge weight. Edges are printed in the order they are added by Prim's algorithm starting from city 0.

The last line prints "Total MST Cost: t", where t is the total integer cost of the minimum spanning tree.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Output: Edge: 0 - 1, Cost: 10
Edge: 3 - 2, Cost: 4
Edge: 0 - 3, Cost: 5
Total MST Cost: 19

Answer

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

// Function to find the vertex with minimum key value not yet included in MST
int minKey(int key[], int mstSet[], int n) {
   int min = INT_MAX, min_index = -1;
   for (int v = 0; v < n; v++) {
      if (mstSet[v] == 0 && key[v] < min) {
         min = key[v];
         min_index = v;
      }
   }
```

```
    return min_index;
}

// Prim's Algorithm
void primMST(int n, int graph[MAX][MAX]) {
    int parent[n];   // Stores MST edges
    int key[n];      // Minimum edge weight to connect each vertex
    int mstSet[n];   // Tracks vertices included in MST

    // Initialize all keys as infinite
    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    // Start from vertex 0
    key[0] = 0;
    parent[0] = -1;

    int totalCost = 0;

    // Build MST with n-1 edges
    for (int count = 0; count < n - 1; count++) {
        int u = minKey(key, mstSet, n);
        mstSet[u] = 1;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    // Print MST edges in the order they were added
    for (int i = 1; i < n; i++) {
        printf("Edge: %d - %d, Cost: %d\n", parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }

    printf("Total MST Cost: %d\n", totalCost);
}
```

```
int main() {
    int n, m;
    scanf("%d %d", &n, &m);

    int graph[MAX][MAX] = {0};

    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        graph[u][v] = w;
        graph[v][u] = w;  // Undirected graph
    }

    primMST(n, graph);
    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Riya is working as a data scientist in a company that deals with large text data. Due to storage and bandwidth limitations, she needs a way to compress text while maintaining its readability.

She decides to use Huffman Encoding, a well-known algorithm that assigns shorter binary codes to frequently occurring characters and longer binary codes to less common characters, thus reducing the overall message size.

Given a set of characters with their respective frequencies, help Riya generate Huffman Codes for each character. Then, using these codes, encode a given message.

Assign '0' to left branches and '1' to right branches, merge the two nodes with the lowest frequency at each step. Clarify that the message will only contain characters from the given input set. Specify that Huffman codes are printed in the order they appear during in-order/pre-order traversal of

the Huffman tree (matching sample output)

The message will only contain characters from the provided character set.

Input Format

The first line of input consists of an integer n, representing the number of unique characters.

The next n lines each consist of a character (an uppercase or lowercase English letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the message to be encoded. The message will only contain characters from the provided character set.

Output Format

The first n lines of output print the Huffman code for each character in the format:

"character: code"

where the characters are printed in the order they appear as leaf nodes during the Huffman tree traversal (left child before right child, assigning '0' to left and '1' to right).

All n input characters and their codes are printed, regardless of whether they appear in the message.

The last line of output prints the encoded message as a sequence of 0s and 1s obtained by replacing each character in the message with its corresponding Huffman code.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
A 1
B 2

C 3
ABCCBAABBC
Output: C: 0
A: 10
B: 11
10110011101011110

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 256

// Huffman tree node
struct MinHeapNode {
    char data;
    unsigned freq;
    struct MinHeapNode *left, *right;
};

// MinHeap structure
struct MinHeap {
    unsigned size;
    unsigned capacity;
    struct MinHeapNode** array;
};

// Create new node
struct MinHeapNode* newNode(char data, unsigned freq) {
    struct MinHeapNode* temp = (struct MinHeapNode*)malloc(sizeof(struct
MinHeapNode));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}

// Create min heap
struct MinHeap* createMinHeap(unsigned capacity) {
    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct MinHeap));
    minHeap->size = 0;
```

```
    minHeap->capacity = capacity;
    minHeap->array = (struct MinHeapNode**)malloc(minHeap->capacity *
sizeof(struct MinHeapNode*));
    return minHeap;
}

// Swap nodes
void swapNode(struct MinHeapNode** a, struct MinHeapNode** b) {
    struct MinHeapNode* t = *a;
    *a = *b;
    *b = t;
}

// Heapify
void minHeapify(struct MinHeap* minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < (int)minHeap->size && minHeap->array[left]->freq < minHeap-
>array[smallest]->freq)
        smallest = left;

    if (right < (int)minHeap->size && minHeap->array[right]->freq < minHeap-
>array[smallest]->freq)
        smallest = right;

    if (smallest != idx) {
        swapNode(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
}

// Extract min
struct MinHeapNode* extractMin(struct MinHeap* minHeap) {
    struct MinHeapNode* temp = minHeap->array[0];
    minHeap->array[0] = minHeap->array[minHeap->size - 1];
    --minHeap->size;
    minHeapify(minHeap, 0);
    return temp;
}
```

```
// Insert node
void insertMinHeap(struct MinHeap* minHeap, struct MinHeapNode*
minHeapNode) {
    ++minHeap->size;
    int i = minHeap->size - 1;
    while (i && minHeapNode->freq < minHeap->array[(i - 1) / 2]->freq) {
        minHeap->array[i] = minHeap->array[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    minHeap->array[i] = minHeapNode;
}

// Build min heap
void buildMinHeap(struct MinHeap* minHeap) {
    int n = minHeap->size - 1;
    for (int i = (n - 1) / 2; i >= 0; i--)
        minHeapify(minHeap, i);
}

// Create and build min heap
struct MinHeap* createAndBuildMinHeap(char data[], int freq[], int size) {
    struct MinHeap* minHeap = createMinHeap(size);
    for (int i = 0; i < size; i++)
        minHeap->array[i] = newNode(data[i], freq[i]);
    minHeap->size = size;
    buildMinHeap(minHeap);
    return minHeap;
}

// Build Huffman tree
struct MinHeapNode* buildHuffmanTree(char data[], int freq[], int size) {
    struct MinHeapNode *left, *right, *top;
    struct MinHeap* minHeap = createAndBuildMinHeap(data, freq, size);

    while (minHeap->size != 1) {
        left = extractMin(minHeap);
        right = extractMin(minHeap);

        top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
```

```
        insertMinHeap(minHeap, top);
    }
    return extractMin(minHeap);
}

// Print codes (recursive traversal)
void printCodes(struct MinHeapNode* root, int arr[], int top, char codes[256]
[256]) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1, codes);
    }
    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1, codes);
    }
    if (!(root->left) && !(root->right)) {
        printf("%c: ", root->data);
        for (int i = 0; i < top; i++) {
            printf("%d", arr[i]);
            codes[(int)root->data][i] = arr[i] + '0';
        }
        codes[(int)root->data][top] = '\0';
        printf("\n");
    }
}

int main() {
    int n;
    scanf("%d", &n);

    char data[n];
    int freq[n];
    for (int i = 0; i < n; i++) {
        scanf(" %c %d", &data[i], &freq[i]);
    }

    char message[257];
    scanf("%s", message);

    struct MinHeapNode* root = buildHuffmanTree(data, freq, n);
```

```
    int arr[MAX], top = 0;
    char codes[256][256] = {0};

    printCodes(root, arr, top, codes);

    // Encode message
    for (int i = 0; i < (int)strlen(message); i++) {
        printf("%s", codes[(int)message[i]]);
    }
    printf("\n");

    return 0;
}
```

Status : Partially correct ***Marks : 7.5/10***

5. Problem Statement

In a clandestine world of spies and covert operations, a group of operatives uses Huffman coding to encode and decode their classified messages securely.

Huffman coding assigns shorter binary codes to more frequently occurring characters and longer codes to less frequent ones. The algorithm works as follows:

Each character and its frequency is inserted into a min-heap (priority queue).At each step, extract the two nodes with the lowest frequencies, merge them into a new internal node whose frequency is the sum of the two, and reinsert it into the heap.Repeat until only one node (the root) remains — this is the Huffman tree.Assign '0' to every left branch and '1' to every right branch. The binary string from root to each leaf node is that character's Huffman code.

Given a set of characters with their frequencies, build the Huffman tree, print the codes for all characters, encode the given message using these codes, and decode the encoded message back to the original.

Note: The message will only contain characters from the provided input

set. All character frequencies are unique.

Input Format

The first line of input consists of a positive integer N, representing the number of unique characters.

The next N lines each consist of a character (an uppercase or lowercase English letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the secret message to be encoded. The message contains only characters from the provided character set

Output Format

The first N lines of output each print the Huffman code for a character in the format:

"character: code"

where characters are printed in the order they appear as leaf nodes during Huffman tree traversal (left child before right child, with '0' assigned to left and '1' to right).

The next line prints the encoded message — a sequence of 0s and 1s obtained by replacing each character in the message with its Huffman code.

The last line prints the decoded message obtained by traversing the Huffman tree using the encoded bits, which must match the original input message.

Refer to the sample outputs for better understanding and formatting.

Sample Test Case

Input: 5
A 5
B 9
C 12

D 13
E 16
ABCDE

Output: C: 00
D: 01
A: 100
B: 101
E: 11
100101000111
ABCDE

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 256

// Huffman tree node
struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
};

// MinHeap structure
struct MinHeap {
    int size;
    int capacity;
    struct Node** array;
};

// Create new node
struct Node* newNode(char ch, int freq) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->ch = ch;
    temp->freq = freq;
    temp->left = temp->right = NULL;
    return temp;
}

// Create min heap
```

```
struct MinHeap* createMinHeap(int capacity) {
    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct MinHeap));
    minHeap->size = 0;
    minHeap->capacity = capacity;
    minHeap->array = (struct Node**)malloc(capacity * sizeof(struct Node*));
    return minHeap;
}

// Swap nodes
void swapNode(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}

// Heapify
void minHeapify(struct MinHeap* minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < minHeap->size && minHeap->array[left]->freq < minHeap-
>array[smallest]->freq)
        smallest = left;

    if (right < minHeap->size && minHeap->array[right]->freq < minHeap-
>array[smallest]->freq)
        smallest = right;

    if (smallest != idx) {
        swapNode(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
}

// Extract min
struct Node* extractMin(struct MinHeap* minHeap) {
    struct Node* temp = minHeap->array[0];
    minHeap->array[0] = minHeap->array[minHeap->size - 1];
    minHeap->size--;
    minHeapify(minHeap, 0);
    return temp;
```

```
}

// Insert node
void insertMinHeap(struct MinHeap* minHeap, struct Node* node) {
    minHeap->size++;
    int i = minHeap->size - 1;
    while (i && node->freq < minHeap->array[(i - 1) / 2]->freq) {
        minHeap->array[i] = minHeap->array[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    minHeap->array[i] = node;
}

// Build min heap
void buildMinHeap(struct MinHeap* minHeap) {
    int n = minHeap->size - 1;
    for (int i = (n - 1) / 2; i >= 0; i--)
        minHeapify(minHeap, i);
}

// Create and build min heap
struct MinHeap* createAndBuildMinHeap(char data[], int freq[], int size) {
    struct MinHeap* minHeap = createMinHeap(size);
    for (int i = 0; i < size; i++)
        minHeap->array[i] = newNode(data[i], freq[i]);
    minHeap->size = size;
    buildMinHeap(minHeap);
    return minHeap;
}

// Build Huffman tree
struct Node* buildHuffmanTree(char data[], int freq[], int size) {
    struct Node *left, *right, *top;
    struct MinHeap* minHeap = createAndBuildMinHeap(data, freq, size);

    while (minHeap->size > 1) {
        left = extractMin(minHeap);
        right = extractMin(minHeap);

        top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
```

```
        insertMinHeap(minHeap, top);
    }
    return extractMin(minHeap);
}

// Print codes and store them
void printCodes(struct Node* root, int arr[], int top, char codes[256][256]) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1, codes);
    }
    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1, codes);
    }
    if (!(root->left) && !(root->right)) {
```

Status : Wrong **Marks : 0/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 27.5

Section 1 : Coding

1. Problem Statement

Prim’s Algorithm for Minimum Spanning Tree (MST):

Sita is planning to lay roads to connect 5 locations in her city. The distances between locations are given as an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to 4).

Help her find the Minimum Spanning Tree (MST) using Prim's Algorithm so that all locations are connected with the minimum total road length. Print each MST edge and its weight in the order they are added to the MST.

Input Format

The input consists of 5 lines, each containing 5 space-separated integers,

representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two locations.

Output Format

The first line of output prints the header: 'Edge \tWeight'

Each of the next V-1 lines prints one MST edge in the format:

'u - v\tw'

where u is the parent location, v is the child location, and w is the integer edge weight, printed in the order edges are added to the MST (MST discovery order).

Refer to the sample output for formatting specifications.

Refer to the sample output for format specifications.

Sample Test Case

Input: 0 2 0 6 0
2 0 3 8 5
0 3 0 8 7
6 8 0 0 9
0 5 7 9 0

Output: Edge Weight
0 - 1 2
1 - 2 3
1 - 4 5
0 - 3 6

Answer

```
#include <stdio.h>
#include <limits.h>

#define V 5   // Fixed number of locations

// Function to find the vertex with minimum key value not yet included in MST
```

```
int minKey(int key[], int mstSet[]) {
    int min = INT_MAX, min_index;
    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

// Prim's Algorithm
void primMST(int graph[V][V]) {
    int parent[V];   // Stores MST edges
    int key[V];      // Minimum edge weight to connect each vertex
    int mstSet[V];   // Tracks vertices included in MST

    // Initialize all keys as infinite
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    // Start from vertex 0
    key[0] = 0;
    parent[0] = -1;

    // Build MST with V-1 edges
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    // Print MST edges
    printf("Edge \tWeight\n");
```

```
    for (int i = 1; i < V; i++) {
        printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
    }
}

int main() {
    int graph[V][V];
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    primMST(graph);
    return 0;
}
```

Status : Partially correct **Marks : 7.5/10**

2. Problem statement

Alex is a network engineer tasked with laying cables to connect multiple offices in the city. Each possible cable connection between two offices has an associated cost. To minimize the total cabling cost while ensuring all offices are connected, Alex decides to use Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the office network.

Given a connected, undirected, weighted graph with V vertices (numbered 0 to V-1) and E edges, find the MST and print each edge included along with the total minimum cost.

Input Format

The first line contains an integer V, representing the number of vertices (numbered 0 to V-1).

The second line contains an integer E, representing the number of edges.

Each of the next E lines contains three space-separated integers u, v, and w, representing an undirected edge between vertex u and vertex v with weight w

Output Format

The first line of output prints: 'Edges in the constructed MST:'

Each of the next V-1 lines prints one MST edge in the format:

'u -- v == weight'

where u and v are vertex numbers (integers) and weight is the edge cost (integer), printed in the order edges are added to the MST (ascending weight order).

The last line prints: 'Minimum Spanning Tree: totalCost'

where totalCost is an integer representing the total MST weight.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
3
0 1 2
1 2 2
0 2 2
Output: Edges in the constructed MST:
0 -- 1 == 2
1 -- 2 == 2
Minimum Spanning Tree: 4

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Structure to represent an edge
typedef struct {
    int u, v, w;
```

```
} Edge;

// Compare function for qsort
int compare(const void* a, const void* b) {
    return ((Edge*)a)->w - ((Edge*)b)->w;
}

// Find function for Union-Find
int find(int parent[], int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent, parent[i]);
}

// Union function for Union-Find
void unionSet(int parent[], int rank[], int x, int y) {
    int xroot = find(parent, x);
    int yroot = find(parent, y);

    if (rank[xroot] < rank[yroot]) parent[xroot] = yroot;
    else if (rank[xroot] > rank[yroot]) parent[yroot] = xroot;
    else {
        parent[yroot] = xroot;
        rank[xroot]++;
    }
}

int main() {
    int V, E;
    scanf("%d", &V);
    scanf("%d", &E);

    Edge edges[E];
    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    // Sort edges by weight
    qsort(edges, E, sizeof(Edge), compare);

    int parent[V], rank[V];
    for (int i = 0; i < V; i++) {
        parent[i] = i;
```

```
        rank[i] = 0;
    }

    int totalCost = 0;
    int edgeCount = 0;

    printf("Edges in the constructed MST:\n");

    for (int i = 0; i < E && edgeCount < V - 1; i++) {
        int u = edges[i].u;
        int v = edges[i].v;
        int w = edges[i].w;

        int setU = find(parent, u);
        int setV = find(parent, v);

        if (setU != setV) {
            printf("%d -- %d == %d\n", u, v, w);
            totalCost += w;
            edgeCount++;
            unionSet(parent, rank, setU, setV);
        }
    }

    printf("Minimum Spanning Tree: %d\n", totalCost);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

You are designing a data compression tool that uses Huffman Coding to encode data efficiently.

Write a program to read a set of characters along with their corresponding Huffman variable-length binary codes, and use them to generate the encoded binary string for a given input message.

It is guaranteed that every character in the input message has a corresponding Huffman code in the provided code table.

Input Format

The first line contains an integer N (integer), representing the number of characters in the code table.

Each of the next N lines contains a character C and its corresponding Huffman code (a binary string of 0s and 1s), separated by a space.

The last line contains the input message (a string of characters) that needs to be encoded. Every character in the message is guaranteed to have a corresponding entry in the code table.

Output Format

The output prints 'Coded Data: ' followed by a binary string (a sequence of 0s and 1s) representing the encoded message, obtained by concatenating the Huffman code of each character in the input message in order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10
D 0100
U 0000
F 0101
M 0001
A 0010
N 0011
C 1100
O 1101
E 1110
H 1111
HUFF

Output: Coded Data: 1111000001010101

Answer

```c
#include <stdio.h>
#include <string.h>

int main() {
    int N;
    scanf("%d", &N);

    char chars[N];
    char codes[N][21];  // Huffman codes up to 20 bits

    // Read code table
    for (int i = 0; i < N; i++) {
        scanf(" %c %s", &chars[i], codes[i]);
    }

    char message[101];
    scanf("%s", message);

    // Encode message
    char result[2001] = "";  // enough space for 100 chars * 20 bits
    for (int i = 0; i < strlen(message); i++) {
        for (int j = 0; j < N; j++) {
            if (message[i] == chars[j]) {
                strcat(result, codes[j]);
                break;
            }
        }
    }

    printf("Coded Data: %s\n", result);

    return 0;
}
```

Status : Correct **Marks : 10/10**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 3

Section 1 : MCQ

1. The frequencies of letters in a file are:

s = 17, k = 24, y = 37

A Huffman tree is constructed, and we assign 0 to the left branch and 1 to the right branch at each node.

Which of the following is the Huffman code for the letter 's'?

Answer

0

Status : Wrong ***Marks : 0/1***

2. In Huffman coding, does data in a tree always occur?

Answer

Leaves

Status : Correct ***Marks : 1/1***

3. Which of the following algorithms is the best approach for solving Huffman codes?

Answer

Brute-force algorithm

Status : Wrong ***Marks : 0/1***

4. From the following given tree, what is the computed codeword for ‘c’?

Answer

101

Status : Wrong ***Marks : 0/1***

5. From the following given tree, what is the code word for the character 'a'?

Answer

011

Status : Correct ***Marks : 1/1***

6. Kruskal’s algorithm is used to _____.

Answer

Traverse the graph

Status : Wrong ***Marks : 0/1***

7. Consider the following graph:

Which edges would be included in the minimum spanning tree using Prim's algorithm starting from vertex A?

Answer

AB, BD, DE, EC, CF

Status : Wrong ***Marks : 0/1***

8. Which of the following is true?

Answer

Prim’s algorithm initializes with a edge

Status : Wrong ***Marks : 0/1***

9. Prim’s algorithm can be efficiently implemented using _____ for graphs with greater density.

Answer

linear search

Status : Wrong ***Marks : 0/1***

10. Which of the following is false about Prim’s algorithm?

Answer

It never accepts cycles in the MST

Status : Wrong ***Marks : 0/1***

11. What is the worst-case time complexity of Prim's algorithm if the adjacency matrix is used?

Answer

O(log V)

Status : Wrong ***Marks : 0/1***

12. Consider a graph G = (V, E), where V = { v1,v2,...,v100 }, E = { (vi, vj) 1≤ i < j ≤ 100} and the weight of the edge (vi, vj) is i−j . The weight of the minimum spanning tree of G is _____.

Answer

100

Status : Wrong ***Marks : 0/1***

13. An undirected graph G(V, E) contains n (n > 2) nodes named v1 , v2 , ...vn. Two nodes vi , vj are connected if and only if 0 < |i − j| <= 2. Each edge (vi, vj) is assigned a weight i + j. A sample graph with n = 4 is shown below. What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes?

Answer

n^2 − n + 1

Status : Correct ***Marks : 1/1***

14. An undirected graph G has n nodes. Its adjacency matrix is given by an n × n square matrix whose (i) diagonal elements are 0's and (ii) non-diagonal elements are 1's. which one of the following is TRUE?

Answer

Graph G has a unique MST of cost n-1

Status : Wrong **Marks : 0/1**

15. Consider a weighted complete graph G on the vertex set {v1,v2 ,v} such that the weight of the edge (v,,v) is 2|i-j|. The weight of a minimum spanning tree of G is:

Answer

nC2

Status : Wrong **Marks : 0/1**

Vel Tech Group of Institutions

Name: KROSURI GANESH
Email: vtu29265@veltech.edu.in
Roll no: vtu29265
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 6_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Anitha is working on packing expensive artifacts into a container that has a limited weight capacity. Each artifact has a weight and a value, and Anitha wants to maximize the total worth of artifacts she can carry, even if it means taking only a fraction of an artifact when the container is nearly full.

The maximum worth is calculated as:

Maximum Worth = Σ (value[i] * fraction_taken[i]),

where fraction_taken[i] = min(1, remaining_capacity / weight[i])

Write a program to ensure that Anitha can calculate the maximum worth of objects she can carry in the container.

Input Format

The first line contains an integer N, representing the number of items.

The second line contains N space-separated integers representing the weights of the items.

The third line contains N space-separated integers representing the values of the items.

The fourth line contains an integer representing the capacity of the container.

Output Format

The output prints a single line showing the total maximum worth of objects carried as a double with exactly two decimal places, in the following format:

"Objects worth Rs.X.YY"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
60 100 120
50
Output: Objects worth Rs.240.00

Answer

```
#include <stdio.h>

// Structure to represent an item
typedef struct {
    int weight;
    int value;
    double ratio;
} Item;
```

```
// Swap function
void swap(Item *a, Item *b) {
    Item temp = *a;
    *a = *b;
    *b = temp;
}

// Sort items by ratio (descending)
void sortItems(Item items[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (items[j].ratio < items[j + 1].ratio) {
                swap(&items[j], &items[j + 1]);
            }
        }
    }
}

int main() {
    int N, capacity;
    scanf("%d", &N);

    int weights[N], values[N];
    for (int i = 0; i < N; i++) scanf("%d", &weights[i]);
    for (int i = 0; i < N; i++) scanf("%d", &values[i]);
    scanf("%d", &capacity);

    Item items[N];
    for (int i = 0; i < N; i++) {
        items[i].weight = weights[i];
        items[i].value = values[i];
        items[i].ratio = (double)values[i] / weights[i];
    }

    // Sort items by value/weight ratio
    sortItems(items, N);

    double totalWorth = 0.0;
    int remaining = capacity;

    for (int i = 0; i < N; i++) {
```

```
        if (items[i].weight <= remaining) {
            // Take whole item
            totalWorth += items[i].value;
            remaining -= items[i].weight;
        } else {
            // Take fraction of item
            totalWorth += items[i].value * ((double)remaining / items[i].weight);
            break;
        }
    }

    printf("Objects worth Rs.%.2f\n", totalWorth);

    return 0;
}
```

Status : Wrong **Marks : 0/10**

2. Problem Statement

Raju is a professional hiker preparing for a long trekking expedition. He has a backpack that can hold a maximum weight of W kilograms. There are N essential items you want to carry, each with a specific weight and importance value (utility score). His goal is to maximize the total importance value of the items you take while ensuring that the total weight does not exceed the backpack's capacity.

He can either include or exclude each item—meaning he cannot take fractions of an item (e.g., you cannot take half of a tent). Given the weights and importance values of the items, determine the maximum possible importance value he can carry in his backpack. Help him to achieve his task.

Input Format

The first line contains two integers, N (number of items) and W (maximum weight the backpack can carry).

The second line contains N integers, representing the weights of the items.

The third line contains N integers, representing the importance values of the items.

Output Format

The output prints a single integer, the maximum importance value that can be achieved without exceeding the weight limit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 5
2 3 4 5
3 4 5 6

Output: 7

Answer

```
#include <stdio.h>

// Utility function to get max of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int N, W;
    scanf("%d %d", &N, &W);

    int wt[N], val[N];

    // Read weights
    for (int i = 0; i < N; i++) {
        scanf("%d", &wt[i]);
    }

    // Read importance values
    for (int i = 0; i < N; i++) {
        scanf("%d", &val[i]);
    }
```

```
    // DP table: dp[i][w] = max value with first i items and capacity w
    int dp[N + 1][W + 1];

    for (int i = 0; i <= N; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            } else if (wt[i - 1] <= w) {
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    // The result is in dp[N][W]
    printf("%d\n", dp[N][W]);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Rohit is responsible for scheduling flights for an airline company. He has a list of N flights numbered from 0 to N-1. Some flights have specific constraints that require them to depart after other flights.

Write a program to help Rohit determine if it's possible to create a flight schedule that adheres to these constraints using Warshall's algorithm.

Example 1

Input:

5

0 1

1 2

2 3

3 4

4 5

Output:

Yes

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

There are no circular dependencies or conflicts in the schedule, so it is possible to create a flight schedule that adheres to the constraints.

Example 2

Input:

6

0 1

1 2

2 3

3 4

4 5

5 0

Output:

No

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

Flight 5 must depart after Flight 0.

There is a circular dependency in the flight constraints. Therefore, it is not possible to create a flight schedule that follows the constraints.

Input Format

The first line of input consists of an integer N, representing the number of flights.

The following N lines consist of two space-separated integers each: the flight number and the flight number that must depart before it.

Output Format

If it is possible to create a flight schedule that adheres to the constraints, print "Yes".

Otherwise, print "No".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 1
1 2
2 3
3 4
4 5
Output: Yes

Answer

```
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    // Relationship matrix (adjacency)
    int graph[N][N];
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            graph[i][j] = 0;
        }
    }

    // Read constraints
    for (int i = 0; i < N; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        graph[a][b] = 1;  // a must depart after b
    }

    // Warshall's Algorithm (transitive closure)
    for (int k = 0; k < N; k++) {
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
            }
        }
    }

    // Check for cycles
    int possible = 1;
    for (int i = 0;
```

Status : Wrong ***Marks : 0/10***

4. Problem Statement

A traveler needs to find the shortest path between every pair of cities in a road network. The cities are connected by roads, each with a specific travel cost. To do this, the traveler uses the Floyd-Warshall Algorithm to calculate

the shortest distances between every pair of cities in a given edge-weighted undirected graph.

Example1

Input:

4

3

0 1 2

1 2 3

2 3 4

Output:

Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Explanation:

Given: 4 nodes (0, 1, 2, 3) and 3 edges: 0 1 (weight 2), 1 2 (weight 3), 2 3 (weight 4)

Any node pair without a direct edge is considered INF (unreachable initially).

Step 1: Construct Initial Graph (Original Matrix)

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Diagonal elements (self to self) are 0.If no direct path exists, it's INF.Step 2: Running Floyd-Warshall Algorithm

Using node 0 as an intermediate: No updates since it only connects to node 1.

Using node 1 as an intermediate: 0 2 via 1: 0 1 (2) + 1 2 (3) = 5

2 0 also updates to 5.

Using node 2 as an intermediate: 0 3 via 2: 0 2 (5) + 2 3 (4) = 9

1 3 via 2: 1 2 (3) + 2 3 (4) = 7

Using node 3 as an intermediate: No further updates.

Step 3: Final Shortest Path Matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Input Format

The first line contains an integer V representing the number of cities (vertices) in the network.

The second line contains an integer E representing the number of roads (edges) in the network.

The next E lines each contain three space-separated integers u, v, w, where u and v represent the cities connected by a road, and w represents the travel cost (weight) of the road.

Output Format

The first line of output prints the original matrix, where each entry represents the

direct travel cost between two cities. If there is no direct road between two cities, represent the cost as INF.

Then, print the shortest path matrix, where each entry represents the shortest travel cost between every pair of cities, computed using the Floyd-Warshall algorithm.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
3
0 1 2
1 2 3
2 3 4
Output: Original matrix
0 2 INF INF
2 0 3 INF
INF 3 0 4
INF INF 4 0

Shortest path matrix
0 2 5 9
2 0 3 7
5 3 0 4
9 7 4 0

Answer

```
#include <stdio.h>

#define INF 999999  // Large value to represent infinity

int main() {
    int V, E;
    scanf("%d", &V);
    scanf("%d", &E);

    int graph[V][V];
```

```
    // Initialize matrix
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (i == j)
                graph[i][j] = 0;
            else
                graph[i][j] = INF;
        }
    }

    // Read edges (undirected graph)
    for (int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        graph[u][v] = w;
        graph[v][u] = w;
    }

    // Print Original matrix
    printf("Original matrix\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (graph[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", graph[i][j]);
        }
        printf("\n");
    }

    // Floyd-Warshall algorithm
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (graph[i][k] != INF && graph[k][j] != INF &&
                    graph[i][k] + graph[k][j] < graph[i][j]) {
                    graph[i][j] = graph[i][k] + graph[k][j];
                }
            }
        }
    }
```

```
    // Print Shortest path matrix
    printf("\nShortest path matrix\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (graph[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", graph[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

A person starts walking from position X = 0, find the probability of reaching exactly on X = N if she can only take either 2 steps or 3 steps. The probability for step length 2 is given i.e. P, and the probability for step length 3 is 1 – P.

Write a program for the same.

Note: Use Dynamic Programming

Example

Input: N = 5, P = 0.20

Output: 0.32

Explanation:

There are two ways to reach 5.

2+3 with probability = 0.2 * 0.8 = 0.16

3+2 with probability = 0.8 * 0.2 = 0.16

So, total probability = 0.32.

Input Format

The input consists of an integer N representing the target position and a double-point number P representing the probability of taking a step of length 2, separated by a space.

Output Format

The output prints a double-point number representing the probability of reaching the exact position N, formatted to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 0.20

Output: 0.32

Answer

```
#include <stdio.h>

int main() {
    int N;
    double P;
    scanf("%d %lf", &N, &P);

    double dp[N+1];
    for (int i = 0; i <= N; i++) dp[i] = 0.0;

    dp[0] = 1.0;  // starting point

    for (int i = 1; i <= N; i++) {
        if (i >= 2) dp[i] += dp[i-2] * P;
        if (i >= 3) dp[i] += dp[i-3] * (1.0 - P);
    }

    printf("%.2f\n", dp[N]);
    return 0;
}
```

Status : Correct *Marks : 10/10*